



UNIVERSIDAD DE MURCIA

FACULTAD DE INFORMÁTICA

TRABAJO FIN DE GRADO

Identificación de ciberataques neuronales contra el cerebro usando inteligencia artificial

Autor:

María Pascual Box

Tutores:

Sergio López Bernal

Gregorio Martínez Pérez

3 de junio de 2024

A mis padres y mis hermanos.

A mis amigos Laura, Pepe, Raquel, Ana al cuadrado, Fernando y David, por hacer más fácil todos estos años de universidad.

A mis tutores y todos los que han ayudado a que el TFG salga adelante, a Sergio, Gregorio, Vicky, Alejandro, y Ángel Luis, por haberse implicado tanto desde el principio.

Índice

	Página
Declaración firmada sobre originalidad del trabajo	v
Resumen	1
Extended Abstract	2
1. Introducción	6
2. Background	8
2.1. Neuronas: sinapsis y potencial de acción	8
2.2. BCI	10
2.3. Machine Learning	13
2.3.1. Deep Learning	14
2.3.2. Red neuronal recurrente	15
2.3.3. Detección de anomalías para series temporales: LSTM Autoencoder	15
3. Estado del Arte	16
4. Objetivos y Metodología	18
4.1. Objetivos	18
4.2. Metodología	18
5. Diseño de la Solución	20
5.1. Simulaciones neuronales	20
5.2. Análisis de datasets	21
5.2.1. Caso espontáneo	21
5.2.2. Caso ataque FLO	23
5.3. Generación de nuevos conjuntos de datos	23
5.3.1. Elección del tamaño de ventana de la serie temporal	24
5.3.2. Creación de datasets	27
5.4. Determinación del modelo LSTM Autoencoder	29
5.4.1. Metodología para determinar el modelo	30
5.4.2. Configuraciones obtenidas	31
6. Análisis de Resultados	33
6.1. Resultados del ataque FLO sobre el 25 % de las neuronas	33
6.2. Resultados del ataque FLO sobre el 50 % de las neuronas	34
6.3. Resultados del ataque FLO sobre el 75 % de las neuronas	35

6.4. Resultados del ataque FLO sobre el 100 % de las neuronas	36
6.5. Análisis comparativo de los resultados	38
7. Conclusiones y Vías Futuras	39

Índice de figuras

1.	Partes de la neurona [30].	8
2.	Proceso del potencial de acción [30].	9
3.	Potencial de acción [20].	9
4.	Funcionamiento general de un BCI.	10
5.	Tecnología DBS [15]	11
6.	Tecnología EEG [42].	12
7.	Morfología de una RNA	14
8.	RNN compuesta por bucles de RNA [29].	15
9.	Diagrama del diseño de la solución.	20
10.	Dataset original procedente de una simulación neuronal en el caso espontáneo.	22
11.	Gráfico que muestra las neuronas que se activan en cada instante de tiempo correspondiente con el dataset de caso espontáneo anterior.	22
12.	Dataset original procedente de una simulación neuronal en el caso de ataque FLO.	23
13.	Series temporales del caso espontáneo para los distintos tamaños de ventana: 50 ms, 100 ms y 200 ms.	25
14.	Visualización de series temporales caso espontáneo (naranja) vs caso ataque (azul) para los tamaños de ventana: 50 ms, 100 ms y 200 ms.	26
15.	Estudio comparativo del tamaño de la ventana de la serie temporal usando boxplot.	27
16.	Esquema de la generación de datos. A la izquierda vemos cómo se forman las permutaciones y a la derecha la información que contiene el dataset en cada permutación.	28
17.	Visualización de la serie temporal de una permutación.	28
18.	Detección del ataque FLO al 25 % de las neuronas sobre el conjunto de prueba.	34
19.	Detección del ataque FLO al 25 % de las neuronas sobre una permutación.	34
20.	Detección del ataque FLO al 50 % de las neuronas sobre el conjunto de prueba.	35
21.	Detección del ataque FLO al 50 % de las neuronas sobre una permutación.	35
22.	Detección del ataque FLO al 75 % de las neuronas sobre el conjunto de prueba.	36
23.	Detección del ataque FLO al 75 % de las neuronas sobre una permutación.	36
24.	Detección del ataque FLO al 100 % de las neuronas sobre el conjunto de prueba.	37
25.	Detección del ataque FLO al 100 % de las neuronas sobre una permutación.	37

Índice de tablas

1. Tabla comparativa de los resultados con las distintas configuraciones de ataque. 38

Declaración firmada sobre la originalidad del trabajo

Dña. **María Pascual Box** con DNI **29528676B** estudiante de la titulación de **Ingeniería Informática** de la Universidad de Murcia y autora del TFG titulado **“Identificación de ciberataques neuronales contra el cerebro usando inteligencia artificial”**.

De acuerdo con el Reglamento por el que se regulan los Trabajos Fin de Grado y de Fin de Máster en la Universidad de Murcia (aprobado C. de Gob. 30-04-2015, modificado 22-04-2016 y 28-09-2018), así como la normativa interna para la oferta, asignación, elaboración y defensa de los Trabajos Fin de Grado y Fin de Máster de las titulaciones impartidas en la Facultad de Informática de la Universidad de Murcia (aprobada en Junta de Facultad 27-11-2015)

DECLARO:

Que el Trabajo Fin de Grado presentado para su evaluación es original y de elaboración personal. Todas las fuentes utilizadas han sido debidamente citadas. Así mismo, declara que no incumple ningún contrato de confidencialidad, ni viola ningún derecho de propiedad intelectual e industrial

Murcia, a 2 de junio de 2024

A handwritten signature in black ink, reading 'María Pascual Box'. The signature is fluid and cursive, with a large loop at the beginning and a horizontal line extending from the end.

Fdo.: María Pascual Box

Autora del TFG

Resumen

Las interfaces cerebro-máquina (BCI) son sistemas bidireccionales que permiten interactuar con el cerebro gracias a la adquisición de datos neuronales, la estimulación de las neuronas o la inhibición de estas. Desde que surgieron en la década de 1970, han tenido un impacto significativo tanto en el ámbito médico, siendo especialmente útiles para el diagnóstico de enfermedades neurodegenerativas o el control de dispositivos externos en la rehabilitación, como en otros campos, entre ellos la autenticación, el entretenimiento y los videojuegos o las aplicaciones basadas en sistemas inteligentes.

Debido a su estrecha relación con el cerebro, surgen muchos problemas en torno a esta tecnología, siendo uno de ellos la ciberseguridad. La mayoría de los estudios sobre la seguridad de estos sistemas son teóricos y carecen de soluciones prácticas. Sin embargo, hay investigaciones que analizan las vulnerabilidades y amenazas de las BCI, incluyendo ciberataques a nivel de neurona individual. De entre todos los ataques propuestos, destacan los ciberataques neuronales, que son aquellos capaces de alterar la actividad neuronal que se da en el cerebro de forma normal.

En este contexto, este trabajo propone una solución para identificar uno de los ocho ciberataques neuronales propuestos en la literatura utilizando técnicas de deep learning y detección de anomalías en series temporales. Para ello, se ha desarrollado un modelo basado en una red LSTM Autoencoder para detectar el ciberataque neuronal FLO, el cual se enfoca en la sobreestimulación de neuronas. El modelo se ha probado en simulaciones derivadas de la corteza visual del ratón, evaluando la actividad cerebral espontánea y bajo ataque, con configuraciones que afectan al 25 %, 50 %, 75 % y 100 % de las neuronas. Así, cada configuración es transformada en una serie temporal y con ellas se generan nuevos conjuntos de datos que serán evaluados por el modelo en la detección de ataques.

El modelo desarrollado demostró ser capaz de predecir y detectar anomalías en la actividad cerebral inducida por el ciberataque neuronal FLO. Los resultados mostraron un porcentaje de detección del ataque del 90 % para la configuración que afecta al 25 % de las neuronas, mientras que para las tres configuraciones restantes, el porcentaje de detección fue del 100 %. A medida que aumenta el grado de ataque, la precisión del modelo para discernir entre la actividad neuronal espontánea y la actividad bajo ataque es cada vez mayor. Estos resultados destacan la eficacia del enfoque propuesto y su potencial aplicación en la protección de sistemas BCI contra ciberataques neuronales.

Extended Abstract

Brain-computer interfaces (BCIs) are bidirectional systems that allow direct interaction with the brain by acquiring neural data, stimulating neurons, or inhibiting them. Since their emergence in the 1970s, these technologies have had a significant impact in various fields. In the medical field, BCI technology has been shown to be particularly beneficial in the treatment of neurodegenerative diseases such as Parkinson's disease and Alzheimer's disease. These systems also play a crucial role in rehabilitation, allowing patients to control the movement of artificial prostheses via neural signals from healthy regions of the brain, thus facilitating recovery and functionality after severe injuries. In addition, they find applications in authentication, entertainment, video games, and applications based on intelligent systems, extending their reach to different aspects of everyday life.

To fully understand the functioning of BCI systems and, by extension, this work, it is necessary to first explain how neurons work at a high level. Neurons are the fundamental units of the brain and nervous system, responsible for receiving, processing, and transmitting information through electrical and chemical signals. This communication process occurs at synapses, where dendrites receive signals from other neurons via neurotransmitters. When these signals exceed a threshold, an action potential, also known as a spike, is generated and propagates along the neuron's axon, triggering the release of neurotransmitters at the next synapse and thus propagating the signal.

There are different types of BCI systems according to their invasiveness. In particular, they can be classified as invasive or non-invasive according to the method of signal acquisition. On the one hand, invasive BCIs are implanted directly into the brain, providing high-quality signals by measuring the activity of individual neurons or groups of neurons, but they present risks such as infection and tissue damage. These technologies include deep brain stimulation (DBS), consisting of an electrode and a pulse generator controlled by a device, and electrocorticography (ECoG) for the acquisition of neuronal activity through electrodes implanted directly in the brain. On the other hand, non-invasive ICBs capture signals without penetrating the scalp, making them safer but with lower resolution. Electroencephalography (EEG) is the most common non-invasive technology due to its low cost and ease of use and is used for the measurement of neuronal activity, while transcranial magnetic stimulation (TMS) is used to stimulate nerve cells in the brain through magnetic fields.

In this context, very ambitious and promising technologies have emerged that aim to achieve neural acquisition and stimulation of the brain at very high resolution at the single neuron level. Two examples of this are the American company led by Elon Musk, Neuralink, and the Australian company Synchron. Both have successfully implanted these devices in the human brain. First Synchron in 2022 and, most recently, Neuralink in January 2024.

Because of their close relationship with the brain, BCI systems pose significant challenges in terms of cybersecurity. The ability to directly manipulate brain activity makes these systems potential targets for attackers, which could lead to serious consequences such as the theft of private information, alteration of cognitive processes, or involuntary control of physical actions. Most studies on the security of these systems are theoretical and lack practical solutions. However, there is research analysing vulnerabilities and threats to BCIs, including cyberattacks at the level of individual neurons. Among all the attacks proposed in the literature, neural cyberattacks stand out, i.e., those capable of altering the neuronal activity that occurs naturally in the brain.

However, there are still no studies in the scientific literature focused on the detection of attacks in the field of neural cybersecurity. In this sense, this work provides a first solution to a problem that has not been addressed so far in the scientific literature, namely the detection of neural cyberattacks. In particular, we propose to identify one of the eight neural cyberattacks proposed in the literature using deep learning techniques and anomaly detection in time series.

To carry out this, a model based on an Autoencoder LSTM network has been developed to detect the Neuronal Flooding (FLO) cyberattack in simulated neural environments. This attack aims to overstimulate several neurons at the same time in a single instant. This model has been tested on simulations derived from the mouse visual cortex, evaluating spontaneous and under attack brain activity with configurations affecting 25 %, 50 %, 75 % and 100 % of neurons. Each configuration is transformed into a time series, generating new data sets that are evaluated by the model for seizure detection.

The solution design was based on simulations provided by the tutors, which consisted of 10 simulations corresponding to spontaneous brain activity and 10 simulations corresponding to the first ones in which the FLO attack occurs at each level of attack. These simulations were based on brain activity from the visual cortex of mice and were constructed using the BMTK tool with the PointNet model for network construction, and the NEST simulator for low-level simulations of the topology, providing a realistic representation of mouse neuronal activity. It is important to note that the creation of these simulations, and therefore the obtaining of these results, are not part of the present work.

The initial analysis of the data sets from the previous simulations focused on understanding the nature of the data and justifying the use of time series. Since the main goal of this work is to identify the FLO neural cyberattack, we focused on detecting neural overstimulations at a given time rather than on identifying specific neurons or the applied voltages. This is why we decided to convert the original data sets, which collect the spikes of the neurons that have been activated and at what instant in time, into a time series that measures the number of spikes occurring in a time interval.

In order to transform the data into a time series, it is very important to select an

appropriate interval length. To this end, several tests were carried out with window sizes of 50 ms, 100 ms, and 200 ms. The results indicated that the 100 ms window size was the best. This window size offered an optimal balance between temporal resolution and the amount of data per window, allowing the model to capture the relevant fluctuations in neural activity without losing critical details or generating excessive noise.

The next step was the generation of new datasets with which to train and evaluate the model for each of the four configurations. On the one hand, in order for the LSTM Autoencoder model to learn, it needs to be trained with many different cases involving variability. However, we only have 10 spontaneous time series and 10 time series with attacks for each test, so we need to generate a large number of spontaneous cases and cases with attacks. To achieve this goal, we generate permutations without repetition, i.e., varying the positions of the data sets without repeating the same one, from 10 data sets, either spontaneous or with attack, but not of different types within the permutation. It should be noted that the permutations are unique, i.e., no two permutations have the same order of data sets, neither for the spontaneous case nor for the case with attack.

On the other hand, we need to learn what naturally occurring activity looks like so that, when an attack occurs, it differs greatly from the neural activity we consider normal. In other words, in order for the LSTM Autoencoder network to detect anomalies, we will have to train it only on normal data and evaluate it on the attack data. Consequently, 80 % of the data will correspond to permutations of the spontaneous data sets and 20 % to permutations of the data sets with attack. In total, 100 permutations were used for each test.

The final part of the solution design focuses on the configuration of the LSTM Autoencoder model for its implementation in FLO attack detection with the different attack configurations. It is important to note that, before proceeding, the data must be scaled in order to be properly used by the model. Subsequently, a simplified three-step methodology is followed to detect attacks in the time series: first, the LSTM Autoencoder network is trained with the data without anomalies; then, the model is used to reconstruct the error in the data with attacks; finally, an error threshold is set and points are labelled as attacks if they exceed this threshold.

The determination of the network architecture and model hyperparameters is carried out by experimenting with a subset of the training set, called the validation set, in order to find the optimal configuration that guarantees the best anomaly detection performance. This process starts by exploring various configurations of the neural network layers, including the number of layers, the number of neurons per layer, and the types of layers. These experiments are conducted with the training configuration set to 15 epochs and a batch size of 100. In addition, the activation function is set to the rectified linear unit (ReLU) and the loss function to the mean square error (MSE). Once the optimal network

configuration has been identified, the next step is to perform a hyperparameter search to further tune the LSTM autoencoder model. This involves varying key hyperparameters, such as the loss function or the activation function, to determine the combination that best improves the performance of the model.

In this context, we are faced with the question of which data set to use in these tests. We have four possible data sets, one for each attack configuration (stimulation at 25 %, 50 %, 75 % and 100 % of the neurons). Since the training data always represents cases with no attacks, the training results should be similar in all four sets, including the threshold set. Variation occurs only in the test results. Stimulation of a certain percentage of neurons only influences the number of spikes during the attack (the higher the percentage, the more spikes), while the threshold remains constant in all cases. Therefore, if the threshold is exceeded by stimulating a certain percentage of neurons, it will also be exceeded by stimulating a higher percentage (in fact, it will be exceeded by more). In other words, if we detect an attack by stimulating a certain percentage of neurons, we will also detect it by stimulating a higher percentage. We therefore chose to test with the data set corresponding to the attack on 25 % of neurons, this being a more realistic representation, as it is unlikely that an attack will reach all neurons in a brain region controlled by a BCI. Finally, we evaluate the model's performance using metrics such as `precision_score`, `recall_score` and `f1_score`, which provide a more complete picture of the model's performance in anomaly detection.

The model developed proved to be able to predict and detect anomalies in brain activity induced by the FLO neuronal cyberattack. The results obtained show a 90 % success rate in detecting the attack for the configuration affecting 25 % of the neurons, while for the remaining three configurations, the detection success rate was 100 %. Moreover, as the degree of attack increases, the accuracy of the model in discerning between spontaneous neuronal activity and activity under attack becomes higher and higher. Although false positives are predicted in all configurations, the high accuracy of the model suggests that these results are manageable and are always preferable to false negatives. All these results suggest that the model is suitable for detecting FLO neural cyberattacks in other configurations where a considerable percentage of neurons are attacked, not just the current ones.

For future work, further tests are planned to assess the effectiveness of the model in detecting the FLO attack when applied to a small percentage of neurons, as well as to explore the evolution of neuron voltage rather than the number of spikes for detection. Furthermore, since only one of the eight neural cyberattacks proposed in the literature has been studied, it is proposed to extend the scope of the work to include the detection of the remaining seven. Finally, it is suggested to explore the use of other machine learning techniques or algorithms in addition to the LSTM Autoencoder model in order to further improve the accuracy and robustness of attack detection.

1. Introducción

Las interfaces cerebro-máquina (BCI) son dispositivos innovadores que permiten interactuar directamente con el cerebro con el objetivo de adquirir la actividad cerebral o realizar acciones de neuroestimulación. La tecnología BCI ha mostrado ser especialmente útil en el tratamiento de trastornos neurológicos como la enfermedad de Parkinson [13] o la enfermedad de Alzheimer [21], utilizando neuroestimulación para mejorar los síntomas y la calidad de vida de los paciente. Además, estos sistemas desempeñan un papel crucial en la rehabilitación, permitiendo a los pacientes controlar el movimiento de prótesis artificiales mediante señales neuronales de regiones sanas del cerebro, facilitando así la recuperación y la funcionalidad tras lesiones graves [34]. En este contexto, en los últimos años han surgido prometedoras y ambiciosas tecnologías como Neuralink o Synchron, presentando los primeros dispositivos BCI con una resolución muy alta, capaces de cubrir poblaciones neuronales pequeñas e, incluso, neuronas individuales [26; 12].

Además de sus aplicaciones médicas, los sistemas BCI también han tenido un impacto significativo en otros escenarios como la autenticación, los juegos o el entretenimiento y aplicaciones basadas en sistemas inteligentes [16]. Sin embargo, a pesar de su potencial y los avances logrados, en términos de ciberseguridad aún queda mucho camino por recorrer. Su capacidad para manipular directamente la actividad cerebral de los usuarios también los convierte en un objetivo para los atacantes, lo que podría derivar en consecuencias graves, como el robo de información privada, la alteración de procesos cognitivos o incluso el control involuntario de acciones físicas. Además de ser escasos, la mayoría de los estudios en la literatura científica abordan la seguridad de estos sistemas desde una perspectiva teórica, sin proporcionar soluciones prácticas y aplicables.

No obstante, existen trabajos que recogen el estado de seguridad de los sistemas BCI, investigando las posibles vulnerabilidades y amenazas a las que pueden estar expuestos. Incluso, encontramos estudios que desarrollan ciberataques en dispositivos BCI con resolución a nivel de neurona individual. En este sentido, López Bernal et al. [4] definieron el concepto de ciberataques neuronales como amenazas capaces de alterar la actividad neuronal espontánea.

Para abordar este problema, este trabajo propone una primera solución para la identificación de ciberataques neuronales mediante técnicas de deep learning y detección de anomalías en series temporales. En concreto, de los ocho ciberataques propuestos en la literatura [5], se ha desarrollado un modelo basado en una red LSTM Autoencoder para detectar anomalías en la actividad neuronal que indique la presencia del ciberataque neuronal FLO, centrado en sobreestimar un conjunto de neuronas. Para ello, se utilizan simulaciones neuronales ya existentes de la corteza visual del ratón en las que se recoge la actividad cerebral espontánea y la misma actividad neuronal en la que se produce el ataque FLO al 25 %, 50 %, 75 % y 100 % de las neuronas. Así, cada configuración es trans-

formada en una serie temporal para poder ser evaluada por el modelo en la detección de ataques.

De esta manera, las aportaciones de este trabajo al estado del arte son dos. Por un lado, cuatro posibles tipos de datasets correspondientes con las cuatro configuraciones de ataque en forma de series temporales. Por otro lado, un procedimiento basado en deep learning y detección de anomalías en series temporales para la detección del ciberataque neuronal FLO.

Los resultados obtenidos han demostrado que el modelo es capaz de identificar con bastante precisión el ataque FLO, detectándolo el 100 % de las veces a partir de la segunda configuración y el 90 % en la primera. A medida que aumenta el grado del ataque, los resultados van mejorando ligeramente, detectando cada vez con mayor precisión si se trata de una actividad cerebral espontánea o de ataque. Sin embargo, también predice falsos positivos en todas las configuraciones, aunque la cantidad de ellos es manejable y siempre son preferibles a falsos negativos. Todo esto sugiere que el modelo es adecuado para detectar el ciberataque neuronal FLO en otras configuraciones, no solo las actuales, donde se ataque sobre un porcentaje considerable de neuronas.

En cuanto a la estructura del documento, este trabajo se divide en siete secciones. La introducción, que pone en contexto al lector sobre la temática del trabajo, la motivación y los resultados obtenidos. El background, donde se explican los conceptos necesarios para entender este trabajo, como neurociencia, sistemas BCI y el aprendizaje computacional. El estado del arte, donde se recogen los trabajos de la literatura científica relacionados con la temática del trabajo. Los objetivos y la metodología, donde se expone tanto el objetivo principal del trabajo como los subobjetivos y los pasos seguidos para llevarlos a cabo. El diseño de la solución propuesta, que incluye el proceso de estudio, toma de decisiones e implementación de las mismas. El análisis de resultados, donde se presentan las soluciones obtenidas con cada una de las configuraciones y una comparativa de ellas. Por último, las conclusiones y las vías futuras, donde se extraen conclusiones de los resultados obtenidos y se describe el futuro trabajo derivado de este trabajo.

2. Background

En esta sección introduciremos todos aquellos conceptos y conocimientos necesarios para poder entender este trabajo. En particular, explicaremos términos de neurociencia y de aprendizaje computacional.

2.1. Neuronas: sinapsis y potencial de acción

La neurona es la célula principal del cerebro y del sistema nervioso cuya función es recibir, transformar y transmitir información a través de señales eléctricas y químicas [45]. Una neurona tiene tres partes fundamentales:

- Dendritas: pequeñas prolongaciones de la neurona que reciben las señales procedentes de otras neuronas.
- Cuerpo celular o soma: encargado de recoger los estímulos procedentes de las dendritas y conducirlos hacia el axón. En él se encuentra el núcleo.
- Axón: encargado de generar, conducir y propagar el impulso eléctrico a las neuronas adyacentes.

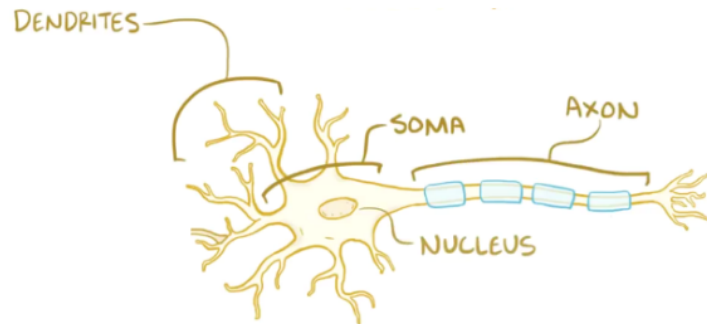


Figura 1: Partes de la neurona [30].

Pasemos a explicar el proceso de comunicación de las neuronas (ver Figura 2), el cual tiene lugar en la sinapsis o espacio entre el extremo de una neurona y la siguiente. En dicho punto, las dendritas reciben señales de otras neuronas mediante neurotransmisores que actúan en forma de señal química. Dicha señal produce la abertura de los canales iónicos, compuertas ubicadas en la membrana de la neurona que se abren ante la presencia de neurotransmisores, cuyo flujo de entrada y salida de iones convierten la señal química en eléctrica. Cuando la carga global de la célula supera un determinado umbral, la neurona activa un potencial de acción, es decir, un impulso eléctrico que recorre el axón, desencadenando la liberación de neurotransmisores en el otro extremo de la célula.

la y retransmitiendo la señal. Estos potenciales de acción son lo que los neurocientíficos denominan spikes [30].

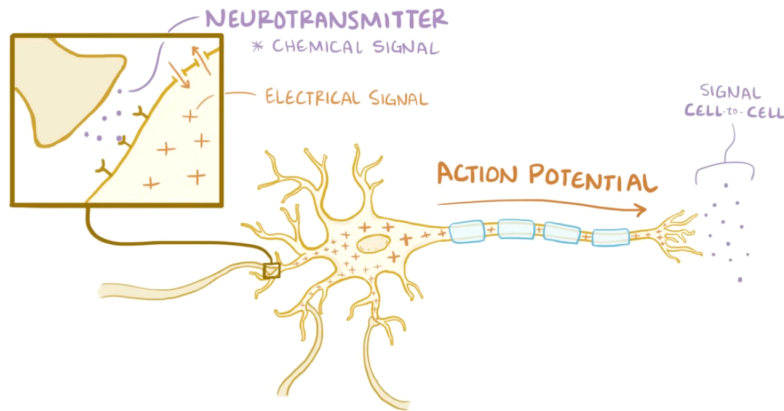


Figura 2: Proceso del potencial de acción [30].

El spike se produce gracias a la diferencia de voltaje entre el exterior y el interior de la neurona. Normalmente, el interior de las neuronas es más negativo que el exterior, con una diferencia alrededor de -70 mV. Cuando los canales iónicos se abren, los iones entran a la neurona para contrarrestar la diferencia de voltaje. Una vez el voltaje de la membrana de la neurona alcanza un determinado valor (entre -50 y -55 mV) conocido como umbral del potencial de acción, es cuando se produce el spike [32]. Por último, cuando el potencial de acción finaliza, los iones regresan al exterior de la neurona hasta que se alcanza el potencial de la membrana inicial.

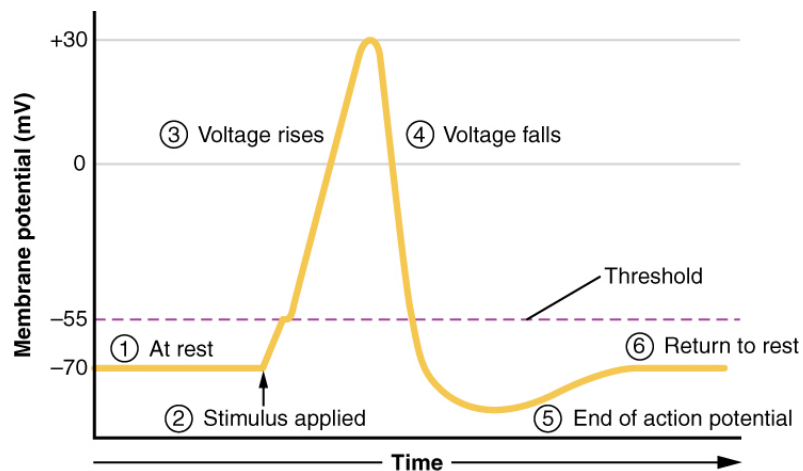


Figura 3: Potencial de acción [20].

2.2. BCI

Una interfaz cerebro-máquina (BCI) es un sistema que permite interactuar directamente con el cerebro. De acuerdo con López Bernal et al. [6], esto se consigue gracias a dos procesos complementarios: la adquisición de la actividad cerebral y la estimulación neuronal.

Como ya hemos visto, las neuronas interactúan entre sí produciendo actividad eléctrica en forma de potenciales de acción. En el proceso de adquisición de la señal, dicha actividad es procesada, codificada y analizada por la BCI, la cual determina la acción pretendida por el usuario. Por último, las aplicaciones ejecutan la acción prevista, permitiendo el control de dispositivos externos. El proceso de estimulación neuronal se lleva a cabo realizando el proceso inverso. La aplicación determina la acción de estimulación a realizar. Luego se codifica dicha acción y la BCI se encarga de estimular las neuronas específicas para llevarla a cabo.

En resumen, un dispositivo BCI puede ser un sistema de comunicación unidireccional o bidireccional entre el cerebro y dispositivos externos. Las comunicaciones unidireccionales se corresponden con la adquisición de la señal o estimulación neuronal, mientras que las comunicaciones bidireccionales se corresponden con la realización de ambas. Esto permite disponer de un funcionamiento adaptativo, en el que el proceso de estimulación se realiza en función del estado del cerebro analizado previamente, además de poder tener un feedback del estado tras estimular.

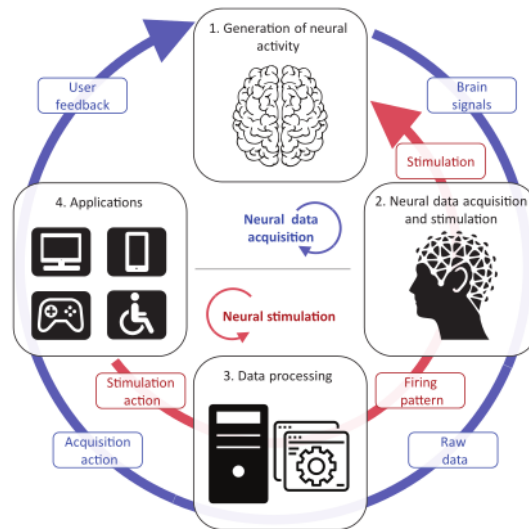


Figura 4: Funcionamiento general de un BCI. La flecha azul se corresponde con la adquisición de la actividad neuronal, mientras que la roja se corresponde con la estimulación neuronal [6].

Los sistemas BCI son ampliamente utilizados en el área de la neuromedicina para ayudar a los pacientes a controlar su cuerpo a través de estos dispositivos con el fin de mejorar los efectos de determinadas enfermedades neurológicas. En este sentido, la tecnología BCI se ha utilizado para el tratamiento de trastornos neurológicos, como la enfermedad de Parkinson [13] o la enfermedad de Alzheimer [21], ambos a través de la neuroestimulación. Otra de las aplicaciones de esta tecnología es la rehabilitación, en la que se controla el movimiento de dispositivos externos como prótesis artificiales mediante señales neuronales de regiones sanas del cerebro [34]. También son utilizados en otros escenarios no médicos como la autenticación de usuario, los juegos y el entretenimiento y las aplicaciones basadas en sistemas inteligentes [16].

Según el método de obtención de la señal, se clasifican en dos tipos: las BCI invasivas y las BCI no invasivas [47; 44]. Las BCI invasivas son implantadas directamente en el cerebro durante una neurocirugía. El dispositivo puede penetrar la corteza cerebral (capa más externa del cerebro) de forma que mida la actividad eléctrica de neuronas individuales, o bien, puede colocarse en su superficie para medir la actividad eléctrica de grupos de neuronas. Los dispositivos invasivos producen señales de mayor calidad en comparación con los BCI no invasivos. Sin embargo, incluyen los efectos secundarios de la cirugía, como posibles infecciones o daños en los tejidos, es más costosa, y una vez implantado, es difícil de solucionar problemas de hardware o de actualizar cualquier componente [48].

Dentro de las BCI invasivas, existen diversas tecnologías. Para la estimulación, destaca la estimulación cerebral profunda (DBS), compuesta por un electrodo y un generador de impulsos controlados por un dispositivo (ver Figura 5). Para la adquisición de la actividad cerebral, encontramos la electrocorticografía (ECoG), en la cual se mide la actividad neuronal gracias a la implantación de electrodos directamente en el cerebro.

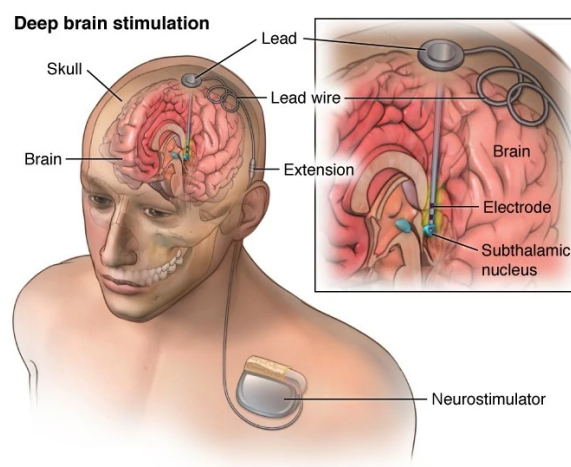


Figura 5: Tecnología DBS [15]

Una de las empresas más prometedoras que desarrollan las tecnologías BCIs invasivas es Neuralink, fundada por Elon Musk en 2016, la cual consiguió implantar con éxito su primer chip en un cerebro humano el 29 de enero de 2024 [3]. Estos sistemas BCI tienen como objetivo obtener la actividad neuronal y estimular el cerebro con una resolución a nivel de neurona individual [26]. Sin embargo, Neuralink no fue la primera en conseguir este hito, fue Synchron en 2022 [43]. Synchron es una empresa australiana caracterizada por desarrollar sistemas BCI mínimamente invasivos. Su aplicación estrella, Stentrode, es un pequeño implante que se inserta en un vaso sanguíneo del cerebro [12]. A pesar de lo ambiciosas y prometedoras que son estas tecnologías, todavía están en desarrollo.

En las BCI no invasivas la señal es captada sin ninguna penetración del cuero cabelludo. Son los dispositivos más utilizados en la tecnología BCI y permiten mejorar ciertas condiciones neurológicas sin la necesidad de intervenciones quirúrgicas, teniendo un efecto acotado en el tiempo, por lo que los efectos secundarios son mucho menores. No obstante, la señal obtenida es la superposición de todas las neuronas que cubre el electrodo del cuero cabelludo (no de neuronas individuales o grupos localizados), por lo que tiene menor resolución.

La tecnología más empleada dentro de las no invasivas debido a su bajo coste, portabilidad y fácil uso es la electroencefalografía (EEG), donde la actividad eléctrica se mide en la superficie del cuero cabelludo mediante varios electrodos (ver Figura 6) que siguen una distribución específica conocida como el “sistema 10-20”. Otras tecnologías son la magnetoencefalografía (MEG) o la imagen por resonancia magnética funcional (fMRI), donde se mide la actividad cerebral mediante campos magnéticos producidos durante la sinapsis o mediante la variación del nivel de oxígeno en sangre, respectivamente [33]. En cuanto a las técnicas de estimulación neuronal, destacamos la estimulación magnética transcraneal (TMS) que utiliza campos magnéticos para estimular las células nerviosas del cerebro [40].

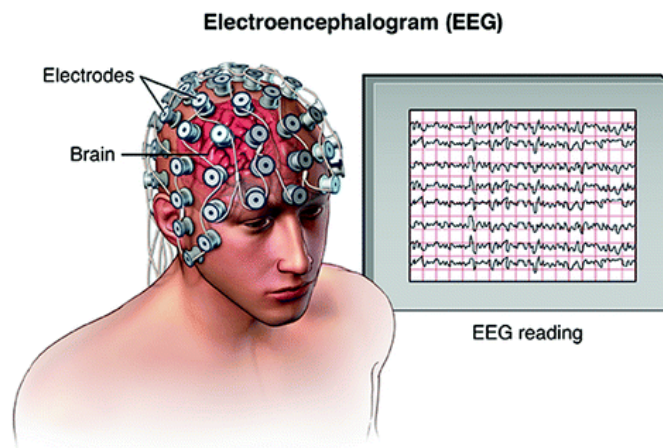


Figura 6: Tecnología EEG [42].

Entre las empresas más conocidas que desarrollan BCI no invasivas encontramos Emotiv [9], cuyos dispositivos EEG portátiles permiten a los usuarios controlar dispositivos y aplicaciones utilizando la actividad cerebral, y Muse [25], que emplea dispositivos EEG portátil enfocados en la meditación y el entrenamiento cerebral.

2.3. Machine Learning

Para resolver un problema en un ordenador, necesitamos un algoritmo. Un algoritmo es una secuencia de instrucciones que deben llevarse a cabo para transformar la entrada en una salida adecuada. Sin embargo, para algunas tareas no disponemos de ningún algoritmo, siendo ejemplos de ello predecir el comportamiento de los clientes, recomendar servicios a cada cliente en particular o distinguir los correos spam de los legítimos. Sabemos cuál es la entrada y cuál es la salida, pero no sabemos cómo transformar la entrada en la salida [2]. En otras palabras, nos gustaría que el ordenador (la máquina) extrajera automáticamente el algoritmo identificando ciertas pautas o regularidades en los datos.

De acuerdo con Géron [11], el aprendizaje computacional (machine learning) es la ciencia y el arte de programar computadoras para que puedan aprender de los datos. Otra definición más general dada por Arthur Samuel en 1959 sería: “El machine learning es el campo de estudio que brinda a las computadoras la capacidad de aprender sin ser programadas explícitamente”.

A partir de los datos, se intenta encontrar el patrón o modelo que mejor prediga el resultado para un nuevo caso de entrada. Sin embargo, la única forma de saber cómo de bien se generalizará un modelo a nuevos casos es probarlo en nuevas observaciones. Por ello, se dividen los datos en dos: en el conjunto de entrenamiento y el conjunto de prueba.

Al evaluar el modelo con el conjunto de prueba, se obtiene el error de generalización, valor que indica cómo de bien funcionará el modelo con instancias que no se han visto antes. Si el error de entrenamiento es bajo, es decir, si el modelo comete pocos errores en el conjunto de entrenamiento, pero el error de generalización es alto, significa que el modelo está sobreajustando los datos de entrenamiento (overfitting). El fenómeno contrario se denomina subajuste (underfitting) y ocurre cuando el modelo es demasiado simple y poco predictivo.

Usando únicamente estos conjuntos no se consigue el mejor modelo posible, pues al medir el error de generalización varias veces en el conjunto de prueba, se adapta el modelo para obtener los mejores resultados en ese conjunto en particular. Por lo tanto, al introducir nuevos datos es poco probable que el modelo funcione bien. Una solución común a este problema es el de reservar una parte del conjunto de entrenamiento, llamado conjunto de validación, para evaluar el modelo con distintos hiperparámetros y seleccionar el mejor. También se pueden usar distintos conjuntos de entrenamiento y validación, estrategia de-

nominada crosvalidación. Por último, se evalúa este modelo final en el conjunto de prueba para obtener una estimación del error de generalización.

Según cómo se supervisan los algoritmos durante el entrenamiento, los modelos de machine learning se clasifican en tres tipos principales: el aprendizaje supervisado, el aprendizaje no supervisado y el aprendizaje por refuerzo. En el aprendizaje supervisado, el conjunto de entrenamiento tiene las soluciones deseadas, llamadas etiquetas, mientras que en el aprendizaje no supervisado los datos de entrenamiento no están etiquetados. Por último, en el aprendizaje por refuerzo el modelo selecciona y realiza acciones a cambio de recompensas o penalizaciones. A continuación, debe aprender por sí mismo cuál es la mejor estrategia para obtener la mayor recompensa a lo largo del tiempo.

2.3.1. Deep Learning

El aprendizaje profundo (deep learning) es el subconjunto de métodos de machine learning basados en redes neuronales artificiales (RNA).

Una RNA está compuesta por unidades conectadas denominados neuronas artificiales, que imitan a las neuronas de un cerebro. Cada neurona artificial recibe señales de sus neuronas conectadas, las procesa y envía señales a otras. Esta señal es un número real, y la salida de cada neurona se calcula mediante una función no lineal de la suma de sus entradas, llamada función de activación. La intensidad de la señal viene determinada por un peso, que se ajusta durante el proceso de aprendizaje. Las neuronas se agrupan en capas. Las señales viajan desde la primera capa o capa de entrada hasta la última capa o capa de salida, normalmente pasando por capas intermedias llamadas capas ocultas [46].

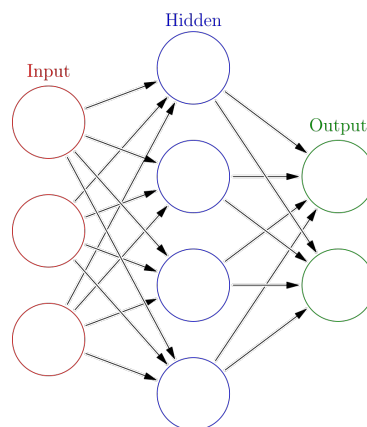


Figura 7: Morfología de una RNA. Cada nodo circular representa una neurona artificial y cada flecha representa una conexión desde la salida de una neurona a la entrada de otra [46].

2.3.2. Red neuronal recurrente

Una red neuronal recurrente (RNN) es un tipo de RNA que incorpora bucles en su arquitectura, permitiendo obtener la información de entradas anteriores para predecir las actuales (ver Figura 8). Son especialmente útiles en el análisis de datos en series temporales, donde la secuencia y el orden de los datos son fundamentales para la predicción o clasificación adecuada.

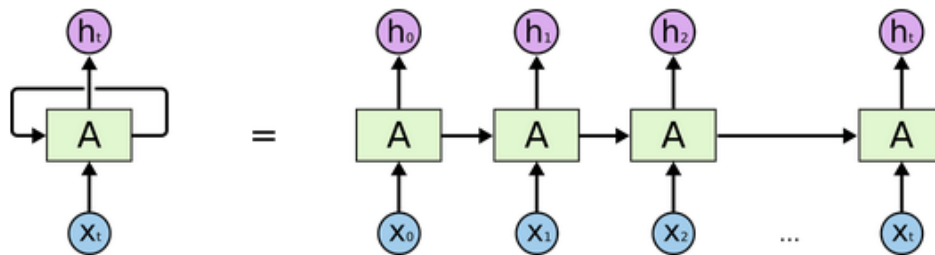


Figura 8: RNN compuesta por bucles de RNA [29].

No obstante, debido a las transformaciones que sufren los datos al pasar por una RNN, se va perdiendo información de las primeras entradas cada paso de tiempo. Para solucionar este problema se introducen celdas de memoria a largo plazo como las celdas GRU o celdas LSTM (red de la cual hablaremos en la siguiente sección) [11].

2.3.3. Detección de anomalías para series temporales: LSTM Autoencoder

Una de las aplicaciones del machine learning es la detección de anomalías para series temporales. Entendemos por serie temporal a una sucesión de datos cronológicamente ordenados que muestran la evolución de una variable en un determinado período de tiempo. Según Hawkins [14], una observación atípica es una observación que se desvía tanto de otras observaciones como para despertar sospechas de que ha sido generada por un mecanismo diferente. Por lo tanto, la detección de anomalías en series temporales consiste en la identificación de elementos atípicos a lo largo del tiempo. Existen diversos modelos con esta finalidad. En este trabajo emplearemos los LSTM Autoencoders.

Por un lado, las redes LSTM (Long Short-Term Memory) son un tipo de RNN capaces de aprender dependencias a largo plazo, lo que las hace particularmente adecuadas para el análisis de datos temporales que evolucionan con el tiempo [29]. Por otro lado, un autoencoder es un modelo de red neuronal que trata de aprender una representación comprimida de una entrada. En concreto, un LSTM Autoencoder es una implementación de autoencoder para datos secuenciales que usa una arquitectura LSTM encoder-decoder de manera no supervisada. Esta arquitectura aprende una representación comprimida de una serie temporal y permite la reconstrucción de esta [7].

3. Estado del Arte

En esta sección presentaremos de forma breve qué se ha hecho en la literatura científica en el ámbito de este trabajo. En concreto, empezaremos exponiendo los estudios que recogen los problemas de ciberseguridad de los sistemas BCI. A continuación, presentaremos artículos sobre ciberataques neuronales y terminaremos exponiendo los trabajos en los que se han empleado redes LSTM para la detección de anomalías.

Desde que surgieron en la década de los 70, en la literatura no se ha considerado la seguridad como un aspecto crítico de las BCI hasta hace pocos años. El primer estudio en identificar las implicaciones de seguridad en los sistemas BCI fue “On the Feasibility of Side-Channel Attacks with Brain-Computer Interfaces” [23] por Martinovic et al. en 2012. En dicho estudio se utilizaron dispositivos BCI de bajo coste basados en EEG para probar la viabilidad de ataques sencillos pero eficaces. Con ello descubrieron que la señal EEG capturada podría revelar la información privada del usuario como tarjetas bancarias, números PIN, lugar de residencia o información sobre personas cercanas.

“Using EEG-based BCI devices to subliminally probe for private information” [10] publicado en 2018 por Frank et al. plantea la posibilidad de efectuar ataques no perceptibles por el usuario (subliminales) utilizando, de nuevo, dispositivos BCI basados en EEG, que comprometen la confidencialidad del usuario. Un ataque se considera subliminal y, por tanto, está por debajo de la percepción cognitiva del usuario, si dura entre 10 y 55 milisegundos. En dicho artículo, basado en una prueba de concepto de 27 participantes, se consiguió demostrar que, mediante un sondeo visual de 13.3 milisegundos y técnicas de clasificación de machine learning, un atacante podría analizar la actividad cerebral subliminal en respuesta a los rápidos estímulos visuales y, en consecuencia, deducir información privada sobre el usuario.

Un artículo interesante que recoge el estado de seguridad en las BCIs es el artículo “Security in Brain-Computer Interfaces: State-of-the-Art, Opportunities, and Future Challenges” [6] publicado en 2020. Además de presentar el estado de seguridad, López Bernal et al. exponen el análisis de los problemas de seguridad de los componentes software que intervienen en los procesos, fases de trabajo y comunicaciones de las BCIs. El trabajo también recopila una gran variedad de ciberataques posibles sobre las BCIs, el impacto que dichas amenazas pueden generar, y posibles estrategias para reducir o mitigar dichos impactos.

Sin embargo, además de ser escasos, la mayoría de los trabajos en la literatura científica abordan la ciberseguridad de los sistemas BCI desde una perspectiva teórica.

Por otro lado, encontramos trabajos que desarrollan ciberataques en dispositivos BCI con resolución a nivel de neurona individual. En “Cyberattacks on Miniature Brain Implants to Disrupt Spontaneous Neural Signaling” [4], López Bernal et al. definieron el

concepto de ciberataques neuronales como amenazas capaces de alterar la actividad neuronal espontánea, es decir, la actividad neuronal que se da en el cerebro de forma normal. Además, estudiaron la potencial aplicación de este tipo de amenazas sobre BCIs de estimulación de nueva generación como Neuralink en base a las vulnerabilidades detectadas por los autores en los diseños propuestos por estas compañías. Este artículo expone los dos primeros ciberataques neuronales, Neuronal Flooding (FLO) y Neuronal Scanning (SCA). FLO es un ataque que tiene como objetivo sobreestimar varias neuronas a la vez en un instante de tiempo determinado, mientras que SCA consiste en estimular las neuronas secuencialmente, impactando solo en una neurona por instante de tiempo. En dicho trabajo, estos ataques son implementados en un simulador neuronal para determinar su impacto sobre el comportamiento neuronal espontáneo, definiendo tres métricas: número de spikes, porcentaje de desplazamientos y dispersión de spikes. Para implementar y validar estos ataques usaron el simulador Brian2 con el modelo neuronal de Izhikevich. En sus resultados vieron que ambos ciberataques producen una reducción en los spikes en comparación con el comportamiento espontáneo, generando un aumento de los desplazamientos temporales y un incremento de la dispersión.

Posteriormente, López Benal et al. publicaron “Eight Reasons to Prioritize Brain-Computer Interface Cybersecurity” [5] en 2022. En el artículo se presenta una taxonomía de ocho ciberataques neuronales que comprenden la estimulación e inhibición de la actividad neuronal, recogiendo los ataques FLO y JAM previamente definidos y exponiendo los seis restantes. Para cada ciberataque se ofrece una definición formal y la conceptualización de su comportamiento, así como un estudio de su impacto a corto y largo plazo.

Desde el punto de vista de la detección de anomalías con redes LSTM Autoencoder, existen numerosos estudios al respecto. Se han usado ampliamente en el área de la medicina para la detección de valores atípicos en el electrocardiograma (ECG) [36; 35; 8] (pudiendo emplearse para la clasificación de arritmias [18]), para la detección de anomalías en datos biomédicos [41] y para la detección de anomalías en imágenes de radiología y patología digital [27], entre otros.

Otro de los campos en los que más son usadas las redes LSTM Autoencoder es el ámbito de la ciberseguridad. En cuanto a la ciberseguridad en red, han sido empleadas para detectar ataques de denegación de servicio (DoS) (intento malicioso de sobrecargar un sistema, servicio o recurso de red con tráfico excesivo para interrumpir o inhabilitar su funcionamiento normal) [39], para la detección de intrusiones en redes informáticas [24] o para evitar que los atacantes se hagan con el control del sistema en las redes de área controlada (CAN) como las que controlan los sistemas del automóvil [19].

No obstante, no existen todavía en la literatura científica trabajos centrados en la detección de anomalías en el ámbito de la ciberseguridad neuronal, por ser un área emergente de estos dos últimos años.

4. Objetivos y Metodología

En esta sección expondremos el objetivo principal del trabajo junto con sus correspondientes subobjetivos y la metodología seguida para alcanzarlos.

4.1. Objetivos

El objetivo principal de este trabajo de fin de grado (TFG) es la detección de ciberataques neuronales mediante técnicas de deep learning en entornos neuronales simulados, en particular el ataque Neuronal Flooding (FLO) con distintos grados de ataque.

Para poder lograr dicho objetivo, se han tenido que alcanzar los siguientes subobjetivos:

- Puesta en contexto con la temática del trabajo, abarcando conceptos fundamentales de neurociencia y de BCI, así como repaso de conceptos de machine learning.
- Revisión del estado del arte relacionado con la ciberseguridad en interfaces cerebro máquina y, más concretamente, de ciberataques neuronales capaces de alterar la actividad neuronal espontánea.
- Revisión de tutoriales para análisis de datos centrados en el uso de las bibliotecas de Python seaborn y pandas.
- Estudio de los datasets proporcionados por los tutores y generación de nuevos conjuntos de datos utilizando Python como lenguaje de programación.
- Lectura de tutoriales acerca de LSTM Autoencoder para detección de anomalías en series temporales.
- Implementar un modelo de detección de anomalías basado en la arquitectura LSTM Autoencoder y realizar pruebas exhaustivas utilizando Python como lenguaje de programación.
- Análisis de los resultados obtenidos, extrayendo conclusiones y explorando posibles vías futuras.

4.2. Metodología

En esta sección detallamos la metodología seguida para alcanzar los objetivos definidos en la sección anterior. La estructura sigue un orden cronológico, detallando cada una de las tareas realizadas para alcanzar cada subobjetivo.

El primer paso fue investigar los conceptos clave para poder realizar este trabajo. Esto supuso estudiar qué son las neuronas, cuál es su estructura, cómo se comunican, qué es la sinapsis y qué son los spikes. También supuso revisar en la literatura qué son los sistemas

BCI, en qué consisten, cómo funcionan y qué tecnologías existen. Además, se llevó a cabo la lectura parcial del libro “Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow” de Aurélien Géron [11] para repasar conceptos clave sobre el aprendizaje computacional, así como aprender otros necesarios para la realización del trabajo.

A continuación, se revisó el estado del arte sobre los artículos relacionados con ciberseguridad en las BCI. En particular, se profundizó en los trabajos realizados por López Bernal et al. [6; 4; 5]. Todos ellos constituyen la base de este trabajo, pues son los que proporcionan el concepto de ciberataque neuronal y exponen los ocho ataques existentes, en concreto el FLO, ataque que buscamos detectar en este trabajo mediante modelos de detección de anomalías en series temporales.

Después, nos familiarizamos con las librerías de Python para análisis de datos, pandas [31] y seaborn [38]. Con estas herramientas, estudiamos los datasets proporcionados por los tutores, obtenidos de una simulación neuronal realista de la corteza visual del ratón, incluyendo 230000 neuronas de diferentes tipos y comportamientos. Estos datasets consistían en cinco tipos diferentes: uno correspondía a la actividad neuronal espontánea, mientras que los otros cuatro representaban distintas configuraciones de ataque, donde se estimulaba al 25 %, 50 %, 75 % y 100 % de las neuronas, respectivamente.

Primero, realizamos un análisis exploratorio de los datos y luego su preprocesamiento. Esto incluyó la transformación de los datos en series temporales, para lo cual fue necesario estudiar el tamaño de la ventana temporal adecuada para agrupar los datos. En esta fase, generamos nuevos conjuntos de datos con las distintas configuraciones de ataque.

Al igual que hicimos con pandas y seaborn, hizo falta documentarse acerca de cómo implementar modelos de redes LSTM Autoencoder para detección de anomalías en series temporales. En particular fueron de utilidad tutoriales como los disponibles en [7; 17], siendo Python el lenguaje de programación empleado.

Una vez ya tuvimos el conocimiento necesario para modelar la red LSTM Autoencoder, el siguiente paso fue determinar su estructura. Para ello, buscamos la mejor configuración de la red en base a ajuste de hiperparámetros. Este fue uno de los pasos más costosos, pues no solo había que determinar los hiperparámetros concretos del modelo, sino que también había que determinar cómo iba a ser la red neuronal. Esto es, cuántas capas iba a tener, de qué tipo, cuántas neuronas por capa, considerar la inclusión de capas dropout y su lugar en la red, entre otros.

Por último, se pasó a ejecutar y evaluar el mejor modelo obtenido en la búsqueda anterior sobre los nuevos conjuntos de datos correspondientes con las diferentes configuraciones de ataque con el fin de poder detectar el ciberataque neuronal FLO en cada uno de ellos.

5. Diseño de la Solución

Esta sección presenta cada uno de los elementos que definen nuestra solución, los cuales quedan resumidos en la Figura 9. Para poder llevarla a cabo, se usan diferentes simulaciones neuronales ya existentes que recrean la actividad neuronal derivada del córtex visual de un ratón. A continuación se estudian los datasets generados por cada una de estas simulaciones, incluyendo la transformación de los mismos en series temporales y la generación de nuevos conjuntos de datos que reflejen las distintas configuraciones de ataque. Estas son: ataque FLO al 25 %, 50 %, 75 % y 100 % de las neuronas, respectivamente. Finalmente se implementa un modelo basado en la arquitectura LSTM Autoencoder para la detección del ciberataque neuronal FLO. Cada uno de estos pasos se analiza con detalle en las siguientes secciones.

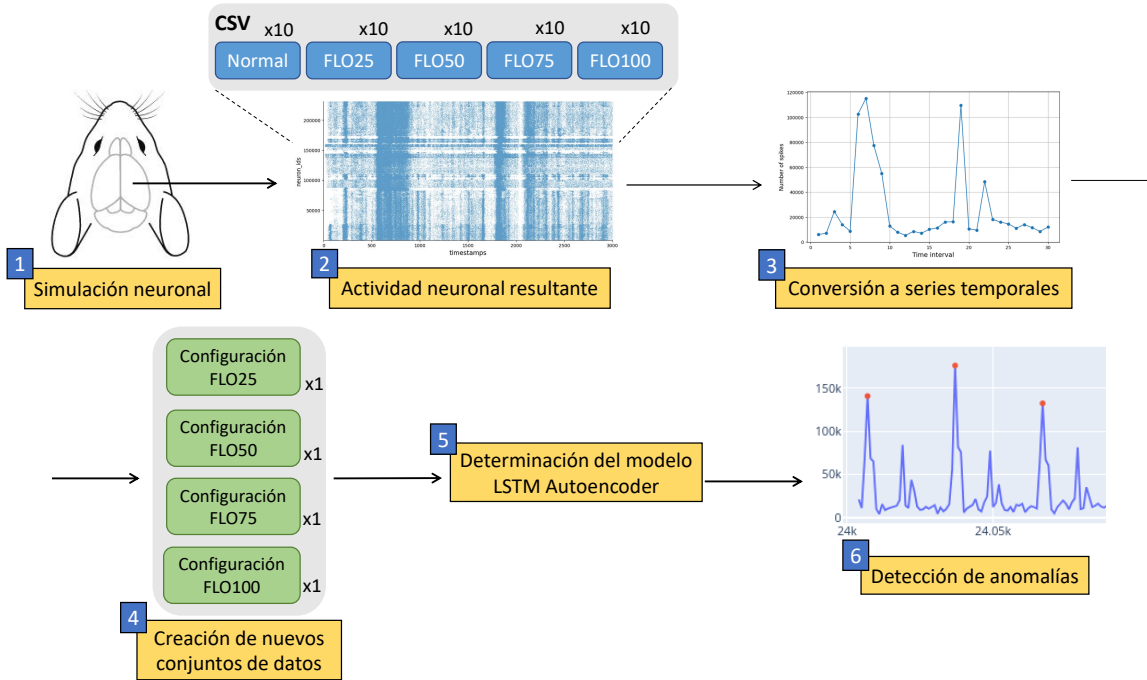


Figura 9: Diagrama del diseño de la solución.

5.1. Simulaciones neuronales

Para poder realizar el trabajo, dispusimos de diferentes simulaciones neuronales proporcionadas por los tutores, en base a un trabajo previo disponible en [22]. Dichas simulaciones se basan en una topología neuronal de la corteza visual primaria de ratones. En concreto, en estas simulaciones se emplea la herramienta BMTK [1], encargada de construir la red haciendo uso del modelo PointNet empleando alrededor de 230.000 neuronas, y se ejecuta

utilizando el simulador NEST [28] para realizar simulaciones de bajo nivel de la topología neuronal.

Los datos recogidos en las simulaciones fueron exportados a ficheros Comma-separated values (CSV). Específicamente, los ficheros de los que dispusimos fueron:

- 10 ficheros CSV procedentes de simulaciones de actividad neuronal espontánea, esto es, cuando no se produce ningún ataque.
- 10 ficheros CSV correspondientes a las simulaciones neuronales anteriores en las que se produce el ataque FLO al 25 % de las neuronas.
- 10 ficheros CSV correspondientes a las primeras simulaciones neuronales en los que se produce el ataque FLO al 50 % de las neuronas.
- 10 ficheros CSV correspondientes a las primeras simulaciones neuronales en los que se produce el ataque FLO al 75 % de las neuronas.
- 10 ficheros CSV correspondientes a las primeras simulaciones neuronales en los que se produce el ataque FLO al 100 % de las neuronas, es decir, se les ataca a todas.

Es importante destacar que la realización de estas simulaciones y, por tanto, la obtención de dichos resultados, no forman parte del presente trabajo.

5.2. Análisis de datasets

Una vez tuvimos los datos, el primer paso fue realizar un análisis de los mismos. Podemos clasificar los datasets en dos tipos: los datasets generados por una actividad neuronal espontánea (caso espontáneo) y los datasets generados por una actividad cerebral que está siendo atacada con el ciberataque neuronal FLO (caso ataque FLO).

5.2.1. Caso espontáneo

Si pasamos a explorar los datasets del caso espontáneo (ver Figura 10) encontramos un total de nueve atributos, todos ellos relacionados con parámetros de configuración para poder llevar a cabo la simulación. Específicamente, los únicos que nos interesan para este trabajo son `attack`, `timestamps` y `neuron_ids`. El atributo categórico `attack` determina el tipo de actividad neuronal simulada, en este caso, espontánea. Los atributos numéricos `timestamps` y `neuron_ids` determinan el instante en ms en el que se ha producido un spike y la neurona que lo ha llevado a cabo, respectivamente.

	attack	n_exec	stimulus	BKG	timestamps	neuron_ids	cortical_layer	population	behaviour
0	Spontaneous	0	Flash_trial_0	BKG_trial_90	565.50	0	VisL5	LIFe5Rbp4	e
1	Spontaneous	0	Flash_trial_0	BKG_trial_90	580.75	0	VisL5	LIFe5Rbp4	e
2	Spontaneous	0	Flash_trial_0	BKG_trial_90	776.75	0	VisL5	LIFe5Rbp4	e
3	Spontaneous	0	Flash_trial_0	BKG_trial_90	2126.75	0	VisL5	LIFe5Rbp4	e
4	Spontaneous	0	Flash_trial_0	BKG_trial_90	1854.25	166745	VisL5	LIFe5noRbp4	e
...	...	...	...	...	...	...	...	...	...
789459	Spontaneous	0	Flash_trial_0	BKG_trial_90	2119.00	97432	VisL4	LIFe4Scnn1a	e
789460	Spontaneous	0	Flash_trial_0	BKG_trial_90	563.25	97434	VisL4	LIFe4Scnn1a	e
789461	Spontaneous	0	Flash_trial_0	BKG_trial_90	1249.25	97434	VisL4	LIFe4Scnn1a	e
789462	Spontaneous	0	Flash_trial_0	BKG_trial_90	2238.00	97434	VisL4	LIFe4Scnn1a	e
789463	Spontaneous	0	Flash_trial_0	BKG_trial_90	2148.50	97442	VisL4	LIFe4Scnn1a	e

Figura 10: Dataset original procedente de una simulación neuronal en el caso espontáneo.

A continuación, realizamos un primer preprocesamiento, quedándonos con estas variables, explorando y visualizando los nuevos datos (ver Figura 11). Como vemos, cada uno de los puntos de la gráfica corresponde con la activación de una neurona (eje Y) en un instante de tiempo dado (eje X) de la simulación. En particular, es interesante destacar que todas las simulaciones tienen una duración máxima de 3000 ms, pues las simulaciones en todos los casos fueron de tres segundos en los experimentos realizados por López Madejska et al. [22].

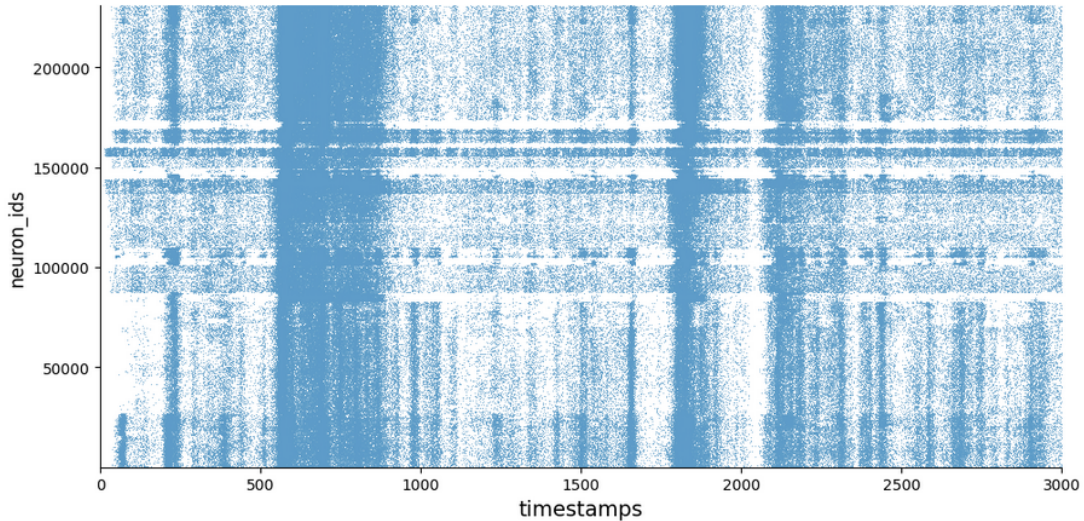


Figura 11: Gráfico que muestra las neuronas que se activan en cada instante de tiempo correspondiente con el dataset de caso espontáneo anterior.

5.2.2. Caso ataque FLO

Recordemos que el ataque FLO tiene como objetivo sobreestimar varias neuronas a la vez en un instante de tiempo determinado. Para ello, debe determinarse el instante de ataque, las neuronas objetivo y el voltaje con que se las quiere estimular.

Si pasamos a explorar los datasets en el caso de ataque FLO (ver Figura 12) encontramos doce atributos, tres de ellos nuevos: `v_increment`, `instant_attack` y `n_neurons`, que se corresponden con los parámetros de configuración del ataque. En concreto, `v_increment` representa el voltaje con el que se estimula a las neuronas, `instant_attack` representa el instante en el que se las ataca y `n_neurons` el número de neuronas que van a ser estimuladas en el ataque. Todas ellas son variables numéricas constantes. Cabe destacar que ahora el atributo categórico `attack` señala el caso de ataque FLO y que el instante de ataque se produce en el instante de 625 ms. Esta vez, los únicos atributos necesarios para el contexto del trabajo son `attack`, `timestamps`, `neuron_ids` e `instant_attack`.

	attack	n_exec	stimulus	BKG	instant_attack	v_increment	n_neurons	timestamps	neuron_ids	cortical_layer	population	behaviour
0	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	632.00	0	VisL5	LIFe5Rbp4	e
1	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	1080.00	0	VisL5	LIFe5Rbp4	e
2	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	1830.00	0	VisL5	LIFe5Rbp4	e
3	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	628.75	162353	VisL5	i5Sst	i
4	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	635.00	162354	VisL5	i5Sst	i
...	...	...	...	...	...	...	...	...	...	...	...	...
838139	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	233.50	99445	VisL4	e4Scnn1a	e
838140	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	563.50	99445	VisL4	e4Scnn1a	e
838141	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	613.50	99445	VisL4	e4Scnn1a	e
838142	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	1816.00	99445	VisL4	e4Scnn1a	e
838143	FLO	0	Flash_trial_4	BKG_trial_94	625	551	230000	2260.00	99445	VisL4	e4Scnn1a	e

Figura 12: Dataset original procedente de una simulación neuronal en el caso de ataque FLO.

5.3. Generación de nuevos conjuntos de datos

Como ya hemos argumentado, el objetivo del TFG es el de identificar el ciberataque neuronal FLO. Para detectarlo, no nos interesa saber qué neuronas han sido estimuladas o con cuánto voltaje se les ha estimulado. En otras palabras, queremos saber si en un instante de tiempo se ha producido una sobreestimulación neuronal respecto a la actividad cerebral que se produce de manera espontánea. Para ello, necesitamos aprender cómo es la actividad que se produce de forma natural para que, cuando se produzca un ataque, esta difiera mucho de la actividad neuronal que consideramos normal.

Es por esto que decidimos convertir el dataset original, el cual recoge los spikes de las neuronas que se han activado y en qué instante de tiempo, en una serie temporal que

mida la cantidad de spikes que se producen en un intervalo de tiempo.

La selección arbitraria del tamaño de la ventana temporal podría afectar significativamente a la capacidad del modelo para capturar patrones relevantes en la serie temporal, haciendo que las predicciones no sean lo más acertadas posible, pudiendo llegar a ser erróneas. En esta sección, explicaremos el proceso de selección del tamaño de ventana de la serie temporal y la transformación de los datos en series temporales para poder entrenar la red LSTM Autoencoder y hacer predicciones.

5.3.1. Elección del tamaño de ventana de la serie temporal

La simulación dura 3000 ms, por lo que la longitud de la ventana temporal tendrá que ser divisor de este número. Un tamaño de ventana demasiado pequeño provocaría que no se capturase la tendencia y la estacionalidad de la serie temporal y un tamaño de ventana muy grande no tendría en cuenta los cambios bruscos (ataques) en la serie temporal. Es por esto que decidimos hacer las pruebas con los tamaños 50, 100 y 200 ms, los cuales se corresponden con 60, 30 y 15 intervalos en la serie temporal. Además, el ataque se producía en el instante 625 ms, por lo que se llevará a cabo en los intervalos 12, 6 y 4, respectivamente.

Empezaremos haciendo las pruebas usando la función `lineplot` de `seaborn` para mostrar las series temporales procedentes de los 10 datasets de caso espontáneo con los tamaños de ventana 50, 100 y 200 ms para tener una primera aproximación de la forma y evolución de la serie temporal (ver Figura 13).

Una primera conclusión que podemos obtener observando la evolución de las series con los distintos tamaños de ventana es que a menor longitud del intervalo tenemos mayor dispersión en el número total de spikes entre los distintos datasets. Si esta dispersión ya aparece en el caso espontáneo, será mucho más difícil para el modelo detectar un ataque, ya que este introduce variabilidad en el número total de spikes.

Por otra parte, buscamos que nuestro modelo de deep learning aprenda los patrones de la actividad neuronal espontánea para que cuando se produzca un ataque lo detecte. Por lo tanto, para seleccionar el tamaño de ventana tendremos que tener en cuenta tanto el caso espontáneo como el caso de ataque. Para el caso espontáneo continuaremos usando los datasets anteriores y para el caso de ataque emplearemos sus 10 simulaciones con ataque FLO en el que se estimula al 50% de las neuronas.

Luego, las siguientes pruebas consistieron en hacer un estudio comparativo para las series temporales del caso espontáneo y del caso de ataque con los distintos tamaños de ventana. Para ello, vemos la evolución de las series temporales del caso espontáneo y del caso de ataque (ver Figura 14) y empleamos la función de `seaborn` `boxplot` para analizar de forma gráfica la dispersión de los datos (ver Figura 15).

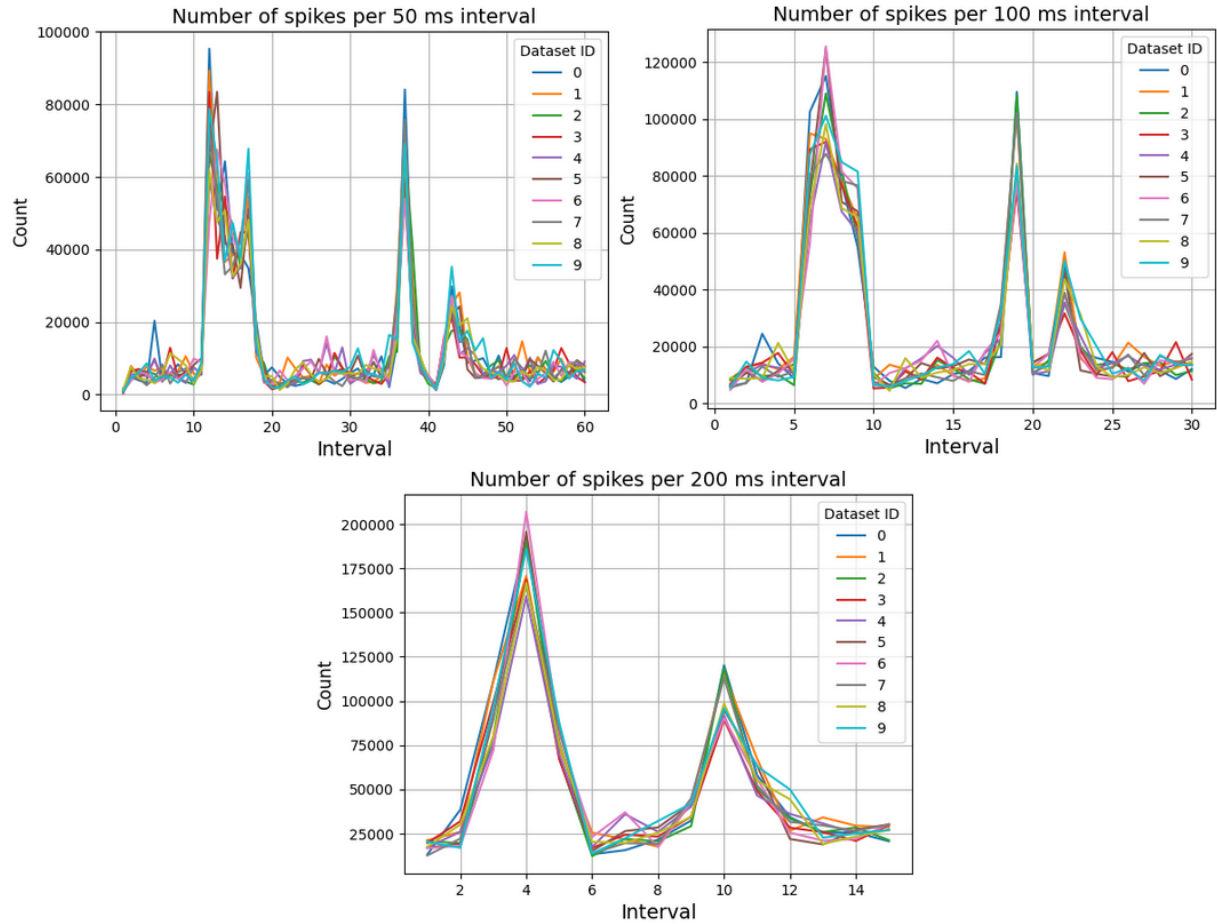


Figura 13: Series temporales del caso espontáneo para los distintos tamaños de ventana: 50 ms, 100 ms y 200 ms.

Si nos fijamos en la evolución de las series (Figura 14), vemos que son idénticas menos el momento en el que se produce el ataque, es decir, el intervalo número 12 cuando el tamaño de la ventana es de 50 ms, el intervalo número 7 para 100 ms, y el intervalo número 4 para 200 ms. Sin embargo, estos gráficos no ofrecen más información que la de visualizar las series temporales en ambos casos.

En cambio, pasemos a observar los boxplot (Figura 15). Con tamaño de ventana 50 ms no solo se muestran diferencias significativas en el momento del ataque (intervalo 12) sino que también en los posteriores, por lo que el modelo de deep learning podría detectar varios puntos de ataque, en vez de uno. Por este motivo y por la dispersión previamente comentada, descartamos 50 ms como la longitud del intervalo en la serie temporal. Por el contrario, para los tamaños de ventana 100 ms y 200 ms encontramos diferencias relevantes en el momento del ataque. Sin embargo, en este instante, las distribuciones de los casos

de ataque y espontáneo se diferencian más con 100 ms que con 200 ms, teniendo en este último puntos en común. Es por esto que la elección del tamaño de ventana para la serie temporal será de 100 ms habiendo, por tanto, 30 intervalos por conjunto de datos.

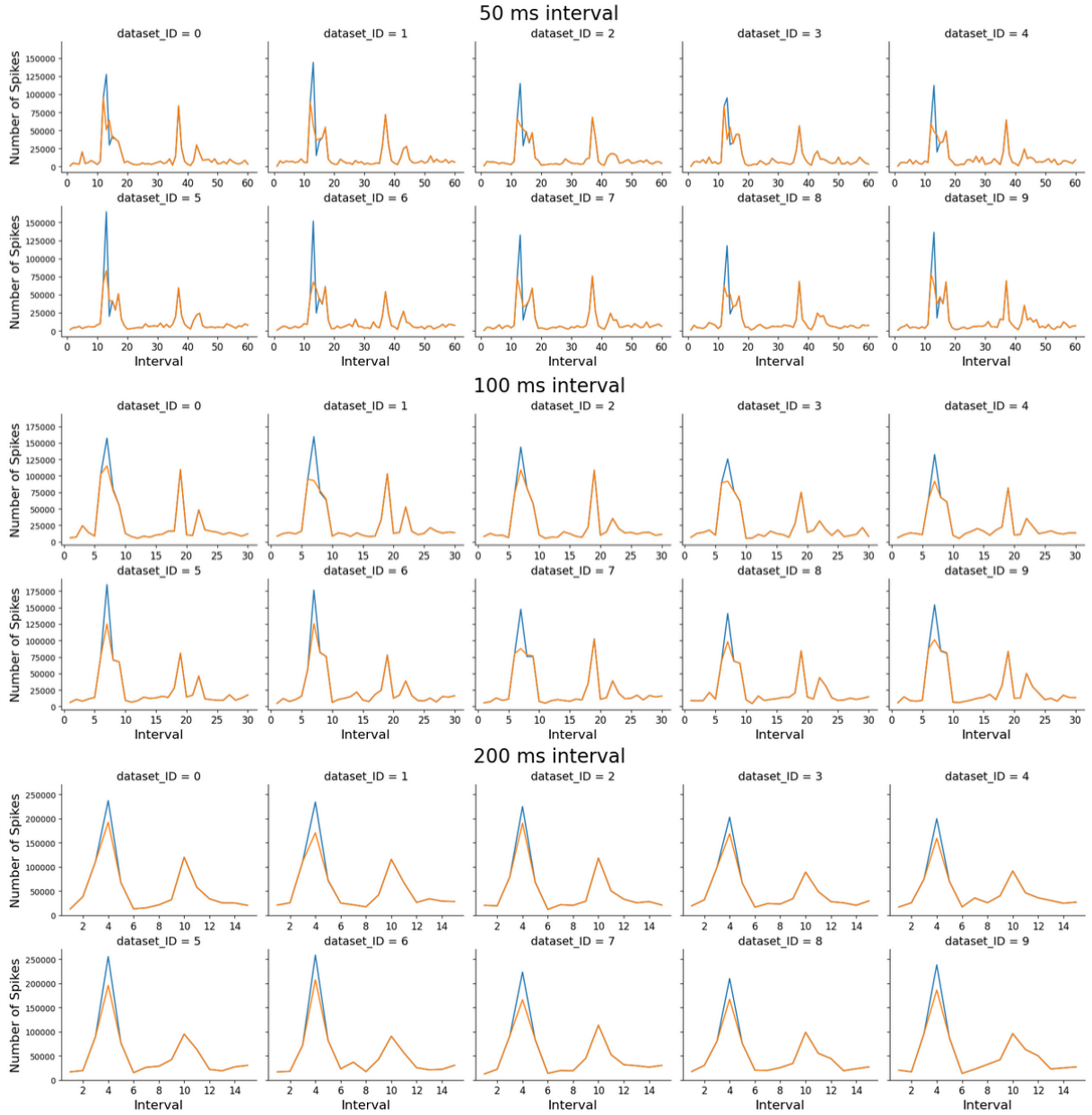


Figura 14: Visualización de series temporales caso espontáneo (naranja) vs caso ataque (azul) para los tamaños de ventana: 50 ms, 100 ms y 200 ms.

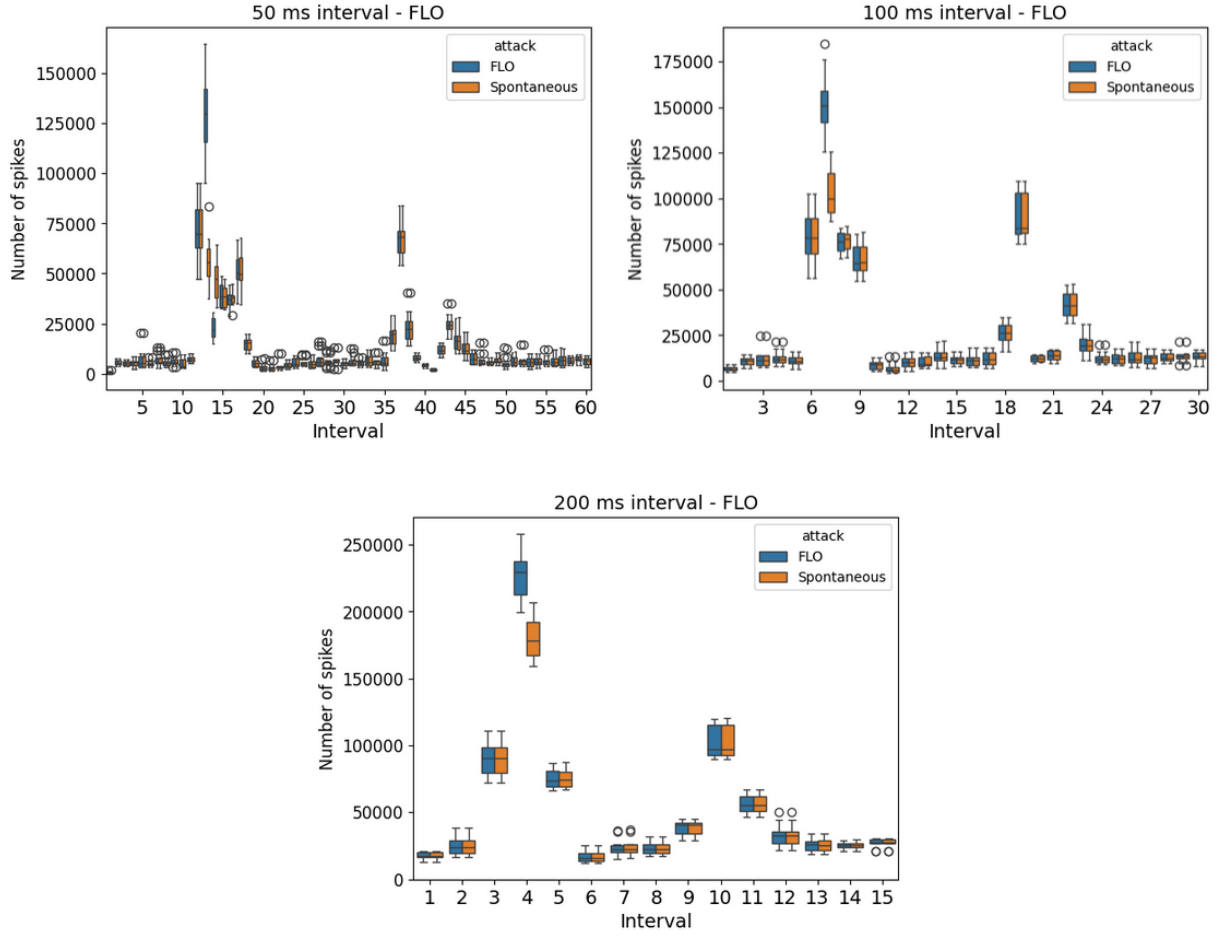


Figura 15: Estudio comparativo del tamaño de la ventana de la serie temporal usando boxplot.

5.3.2. Creación de datasets

Necesitamos varios casos diferentes que supongan variabilidad para entrenar el modelo. Sin embargo, solo disponemos de 10 series temporales espontáneas y 10 series con ataques para cada prueba, por lo que necesitamos generar una gran cantidad de casos espontáneos y casos con ataque. Para conseguir este objetivo, seguiremos la siguiente estrategia: enumerar los datasets del 0 al 9 y hacer permutaciones sin repetición (variar las posiciones de los datasets sin repetir el mismo) con los 10 datasets (ver Figura 16). De este modo tendríamos 10 posibles opciones para poner en el primer lugar, 9 posibles datasets para poner en el segundo, 8 para el tercero, y así sucesivamente. En total tendríamos un total de $10!$ permutaciones.

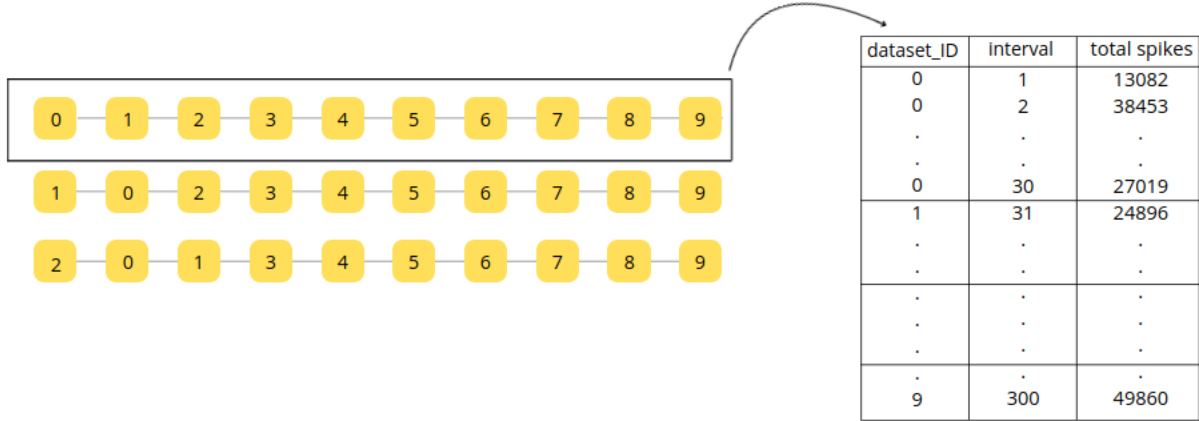


Figura 16: Esquema de la generación de datos. A la izquierda vemos cómo se forman las permutaciones y a la derecha la información que contiene el dataset en cada permutación.

Además, cada permutación tendrá el identificador del dataset, el cual determina el orden dentro de la propia permutación, y la serie temporal formada por la unión de las 10 series temporales correspondientes con cada dataset. En este sentido, como cada dataset tiene una serie temporal de 30 intervalos, la serie temporal de la permutación tendrá 300 intervalos. Un ejemplo de cómo quedaría una permutación sería la Figura 17.

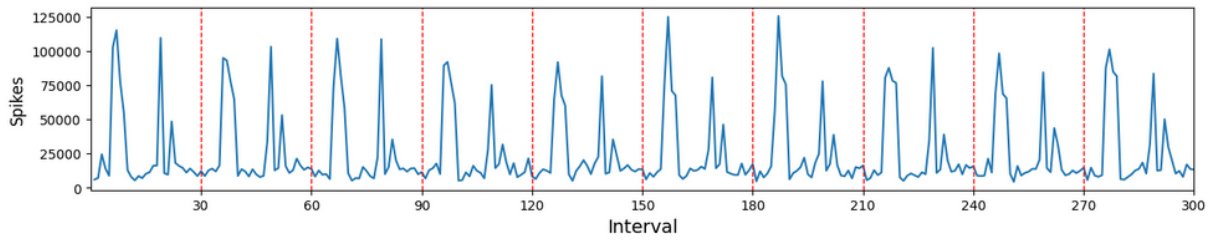


Figura 17: Visualización de la serie temporal de una permutación.

Por un lado, para que una red LSTM Autoencoder detecte anomalías tendremos que entrenarla únicamente con datos normales y evaluarla con los datos que presenten ataques. Así, aprenderá la estructura interna de los datos, las dependencias a largo plazo y la estacionalidad de la serie e identificará las anomalías (ataques) detectando las desviaciones significativas en la reconstrucción de los datos. En consecuencia, el 80 % de los datos se corresponderán con permutaciones de datos espontáneos y el 20 % con permutaciones de datos con ataque. Destacar que las permutaciones son únicas, es decir, un mismo orden de los datasets en una permutación de datos espontánea es distinta de todas las permutaciones de datos espontáneos y de todas las permutaciones de datos con ataque. Lo mismo sucede con las permutaciones procedentes de simulaciones con ataque.

Por otro lado, con $10!$ permutaciones tendríamos una cantidad ingente de datos, pues por cada permutación hay 300 ventanas en la serie temporal, lo que haría un total de $10! * 300 = 1,088,640,000$ datos. Por ello, haremos las pruebas con un conjunto de datos mucho menor. En particular, usaremos 100 permutaciones, o equivalentemente, $100 * 300 = 30,000$ intervalos por dataset.

Teniendo en cuenta cómo se forman las permutaciones, cuántas vamos a seleccionar, y qué porcentaje de ellas usan las simulaciones con actividad cerebral espontánea y cuántas con ataque, generaremos un total de cuatro datasets, uno por cada ataque FLO. En este sentido, los ficheros generados serán:

- Dataset formado por 100 permutaciones con las primeras 80 permutaciones de casos espontáneos y 20 permutaciones de ataque FLO que estimula al 25 % de las neuronas.
- Dataset formado por 100 permutaciones con las primeras 80 permutaciones de casos espontáneos y 20 permutaciones de ataque FLO que estimula al 50 % de las neuronas.
- Dataset formado por 100 permutaciones con las primeras 80 permutaciones de casos espontáneos y 20 permutaciones de ataque FLO que estimula al 75 % de las neuronas.
- Dataset formado por 100 permutaciones con las primeras 80 permutaciones de casos espontáneos y 20 permutaciones de ataque FLO que estimula al 100 % de las neuronas.

5.4. Determinación del modelo LSTM Autoencoder

En esta última sección determinaremos la configuración del modelo LSTM Autoencoder para poder usarla en la detección del ciberataque neuronal FLO en los distintos datasets. Pero antes, necesitamos entender, de forma general, cómo funciona esta arquitectura para llevar a cabo la detección de anomalías. Después explicaremos la metodología seguida para determinar el modelo y, por último, las configuraciones obtenidas tras aplicar este procedimiento.

Para utilizar un LSTM Autoencoder, es necesario escalar previamente los datos. Una vez normalizados, el modelo LSTM Autoencoder espera como entrada un array tridimensional con dimensiones [samples, time_steps, features], donde samples es el número total de ejemplos en el conjunto de datos (es decir, las instancias de la serie temporal que se utilizarán para entrenar la red), time_steps es el número de datos de la serie temporal utilizados para predecir el siguiente valor, y features es el número de características que se busca predecir. La secuencia de salida debe ser un array unidimensional que contenga los valores a predecir.

De forma muy resumida y sin entrar en detalles, los pasos que seguiremos para detectar los ataques en las series temporales son:

1. Entrenar el modelo LSTM Autoencoder con los datos sin anomalías, es decir, los del caso espontáneo.
2. Usar el LSTM Autoencoder para reconstruir el error en los datos con ataques.
3. Si el error de reconstrucción para los datos de prueba está por encima de un umbral establecido (threshold), etiquetamos el punto correspondiente como ataque.

Una metodología común en detección de anomalías para determinar el threshold se conoce como método de la desviación estándar [37]. Dada la distribución de los datos, tomamos la media μ y la desviación estándar σ y consideramos el rango $(\mu - 3\sigma, \mu + 3\sigma)$. Entonces el 99.7 % de los datos se encuentran dentro de este rango y cualquier valor fuera se considera anomalía. Además, como estamos buscando una cota superior, tomaremos como threshold el máximo de este intervalo, es decir, $\mu + 3\sigma$.

5.4.1. Metodología para determinar el modelo

En primer lugar, tomaremos un subconjunto del conjunto de entrenamiento que llamaremos conjunto de validación. A continuación, determinaremos la forma de la red neuronal recurrente, esto es, número de capas, neuronas por capa, tipo de capa, así como la posible inclusión de capas dropout, con aquella configuración que mejores resultados obtenga con ese conjunto de validación. Estos experimentos los haremos fijando las configuraciones de entrenamiento de 15 épocas y tamaño de batch size de 100, así como los parámetros del modelo: la función de activación será la unidad lineal rectificada (ReLU), la función de pérdida será el error medio cuadrático (MSE) y el número de time_steps será igual a 3.

Una vez ya sabemos la configuración de la red neuronal recurrente, el siguiente paso consiste en una búsqueda de hiperparámetros para determinar los hiperparámetros del modelo LSTM Autoencoder (time_step, función de pérdida, función de activación). De nuevo, estos experimentos solo se hacen con el conjunto de validación y, solo cuando ya tengamos la mejor configuración de la red y de hiperparámetros posibles, lanzaremos las pruebas para la detección de anomalías sobre el conjunto de test.

La siguiente cuestión es, ¿con qué datasets hacemos estas pruebas? Recordemos que tenemos 4 posibles conjuntos de datasets, uno por cada posible configuración de ataque (estimulación al 25 %, 50 %, 75 % y 100 % de las neuronas). Puesto que los datos de entrenamiento son siempre casos sin ataque, los resultados del entrenamiento deben ser muy parecidos con los 4 conjuntos (incluido el umbral establecido). Luego solo varían los resultados en el test. La estimulación a x % de las neuronas solo afecta a la cantidad de spikes durante el ataque (a mayor porcentaje, mayor cantidad de spikes) mientras que el threshold es equivalente en todos los casos. Por lo que, si se supera el umbral estimulando al x % de las neuronas, se superará el umbral estimulando al (x+1) % (de hecho, se sobrepasará por más). En otras palabras, si detectamos un ataque estimulando al x %, lo detectaremos

también con el $(x+1)\%$. Por lo tanto, decidimos hacer las pruebas con el dataset de ataque al 25 % de las neuronas, lo cual resulta más realista, pues es muy poco probable que un ataque sea capaz de cubrir la totalidad de neuronas de una región cerebral controlada por una BCI.

En cuanto a las métricas de rendimiento para evaluar el modelo final con las distintas configuraciones de ataque, usaremos `precision_score`, `recall_score` y `f1_score` de la biblioteca Scikit-Learn de Python. `Precision_score` mide la proporción de datos espontáneos que fueron detectados correctamente como espontáneos, `recall_score` mide la proporción de ataques que fueron etiquetados correctamente como ataques y `f1_score` es una métrica que combina las dos anteriores. Estas métricas se calculan a partir de verdaderos positivos (TP), es decir, puntos detectados como anomalías que son ataques; falsos positivos (FP), es decir, puntos detectados como anomalías que no son ataques; falsos negativos (FN), estos son, puntos no detectados como anomalías que son ataques. Además, resulta interesante introducir el concepto de verdaderos negativos (TN), que corresponden a puntos no detectados como anomalías que son normales, y que posteriormente serán de utilidad en la Sección 6. Las fórmulas utilizadas para obtener estas métricas son:

$$\text{precision} = \frac{TP}{TP + FP}$$

$$\text{recall} = \frac{TP}{TP + FN}$$

$$\text{F1-score} = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

Decidimos no usar `accuracy_score`, métrica que representa el porcentaje total de valores correctamente clasificados, puesto que la cantidad de datos normales es mucho mayor que la de datos anómalos. En otras palabras, al estar la proporción de datos desbalanceada, sin identificar ningún ataque ya obtendríamos un `accuracy` por encima del 96 % (pues solo hay un ataque por cada dataset, es decir, cada 30 datos de la serie temporal). Es por esto que usamos `f1_score`.

5.4.2. Configuraciones obtenidas

Tras aplicar la metodología previamente descrita, obtuvimos la siguiente arquitectura del modelo LSTM Autoencoder:

```
model = Sequential()  
model.add((LSTM(32, return_sequences=True, activation='relu')))  
model.add(Dropout(rate=0.1))
```

```
model.add((LSTM(16, return_sequences=True, activation='relu')))  
model.add((LSTM(8, return_sequences=True, activation='relu')))  
model.add(Dropout(rate=0.1))  
model.add((LSTM(4, return_sequences=False, activation='relu')))  
model.add(Dense(X_train.shape[2]))  
optimizer = keras.optimizers.Adam(learning_rate=0.01)  
model.compile(optimizer=optimizer, loss="mse")
```

Como vemos, usamos una primera capa de entrada de tipo LSTM con 32 neuronas, 3 capas LSTM ocultas con 16, 8 y 4 neuronas respectivamente, y una capa de salida densa, además de dos capas Dropout para prevenir el overfitting. Como función de activación ReLU y como función de pérdida MSE.

El otro parámetro que había que determinar fue `time_step`, es decir, el tamaño de la ventana que usa la red LSTM para predecir el siguiente valor. Puesto que las anomalías se encontraban en el séptimo intervalo dentro de cada dataset, probamos con `time_step` menor que 7, ya que si probamos con un valor igual o superior a 7, el primer ataque de todos no lo detectaría. Finalmente, obtuvimos que el mejor valor era para `time_step` igual a 3.

6. Análisis de Resultados

En esta sección mostraremos los resultados obtenidos tras aplicar el modelo LSTM Autoencoder para detectar el ciberataque neuronal FLO en cada una de las cuatro configuraciones de ataque. Dichas configuraciones se corresponden con la sobreestimulación neuronal del 25, 50, 75 y 100 % de las neuronas.

Para cada uno de los ataques, haremos un estudio cuantitativo y mostraremos gráficas sobre los resultados. El estudio cuantitativo consistirá en el estudio de las métricas de rendimiento junto con el número de anomalías que detecta. Recordemos que los datasets contienen 20 permutaciones con ataque, lo que equivale a 200 datasets y, por tanto, 200 anomalías. Además, cada dataset contiene 30 intervalos y el ataque se producía en el número 7, lo que equivale gráficamente al primer máximo de los dos que tiene cada dataset. Las gráficas, en cambio, se corresponderán con la detección de ataques por parte del modelo, esto es, las anomalías en el número de spikes de la serie temporal. Finalmente, realizaremos un análisis comparativo de los resultados logrados.

6.1. Resultados del ataque FLO sobre el 25 % de las neuronas

Los resultados obtenidos por las métricas de rendimiento estimulando al 25 % de las neuronas han sido:

Anomaly points: 200
Precision: 0.9
Recall: 0.9
F1-score: 0.9

Como vemos, se detectan 200 puntos como anomalías. Sin embargo, tanto precision como recall toman el valor de 0.9. Esto implica que de los 200 puntos detectados como anomalías, 180 son verdaderos positivos, es decir, ataques predichos correctamente, mientras que 20 son falsos positivos, esto es, puntos detectados como anomalías que son normales. Además, existen aproximadamente 20 puntos no detectados como anomalías que son ataques (falsos negativos) y el resto son puntos normales predichos correctamente (verdaderos negativos). Por otra parte, f1_score toma el valor de 0.9, lo cual indica que el modelo predice correctamente los puntos (si son ataques o no) la mayoría de las veces.

A continuación, mostramos las gráficas correspondientes con la predicción del modelo sobre el conjunto de prueba (ver Figura 18) y sobre una permutación (ver Figura 19). Si observamos las dos figuras anteriores, notamos que la primera resulta poco informativa debido a la gran cantidad de puntos superpuestos, lo que dificulta la visualización de la función. En cambio, la segunda figura se corresponde con la detección de ataques en una permutación cualquiera. Vemos que de los 10 ataques que debería identificar (el primer

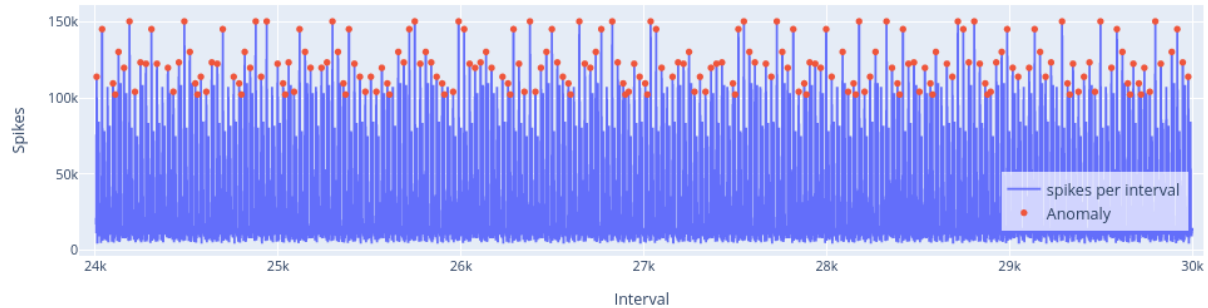


Figura 18: Detección del ataque FLO al 25 % de las neuronas sobre el conjunto de prueba.

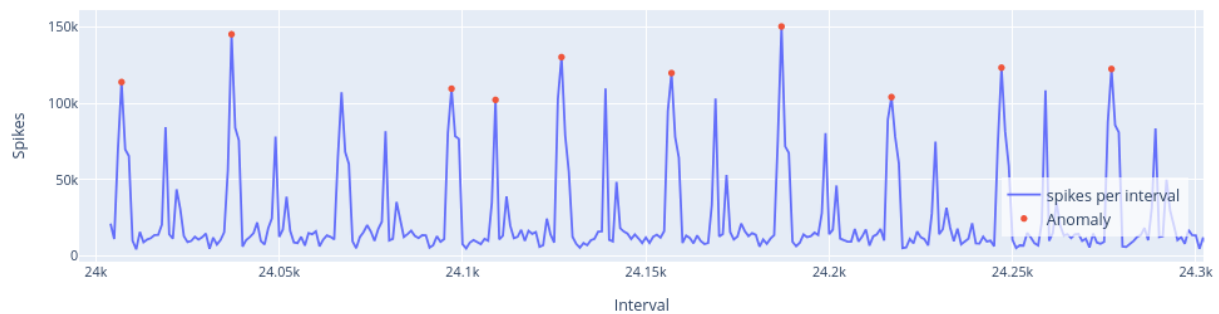


Figura 19: Detección del ataque FLO al 25 % de las neuronas sobre una permutación.

máximo de cada dos), determina 9 correctamente y 1 incorrectamente (la cuarta anomalía empezando por la izquierda).

6.2. Resultados del ataque FLO sobre el 50 % de las neuronas

Los resultados obtenidos por las métricas de rendimiento estimulando al 50 % de las neuronas han sido:

Anomaly points: 240

Precision: 0.833

Recall: 1.0

F1-score: 0.909

Se detectan 240 puntos como anomalías, la precisión obtenida es de 0.833 y el recall de 1.0. Esto implica que de los 240 puntos detectados como anomalías, 200 son verdaderos positivos, mientras que 40 son falsos positivos. Además, no hay falsos positivos y el resto de puntos son verdaderos positivos. En otras palabras, el modelo predice más anomalías que ataques reales, pero detectando siempre los ataques. De nuevo, f1_score toma un valor alto, lo que implica que el modelo sigue prediciendo correctamente los puntos en la

gran mayoría de los casos.

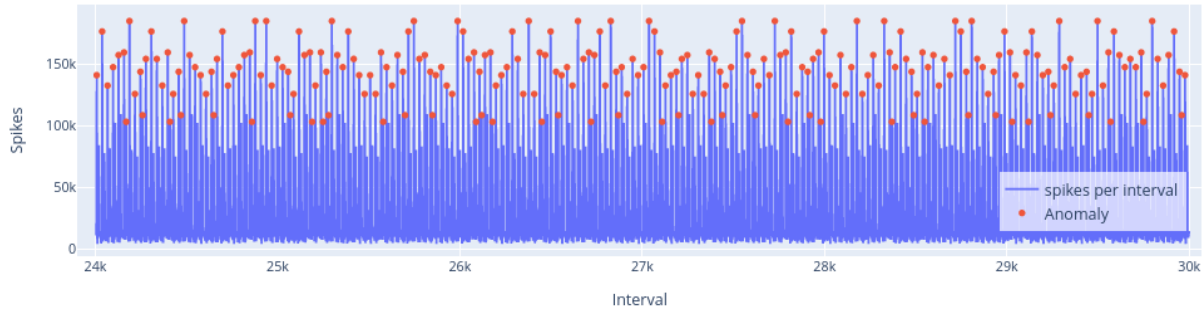


Figura 20: Detección del ataque FLO al 50 % de las neuronas sobre el conjunto de prueba.

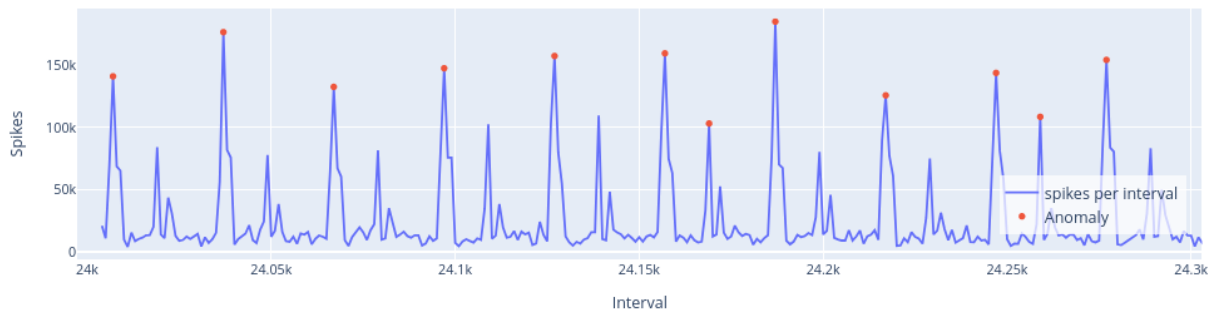


Figura 21: Detección del ataque FLO al 50 % de las neuronas sobre una permutación.

Ahora, mostramos los gráficos correspondientes sobre la predicción del modelo (ver Figura 20 y Figura 21). Observando el primer gráfico, notamos que el número de spikes por intervalo durante el ataque es significativamente mayor que en el caso anterior, ya que esta vez se estimula al 50 % de las neuronas. En el segundo gráfico vemos que se detectan 12 puntos como anomalías, de los cuales 10 corresponden a ataques predichos correctamente (el primer máximo de cada dos), mientras que los 2 restantes son puntos espontáneos que se identificaron erróneamente como anomalías (falsos positivos).

6.3. Resultados del ataque FLO sobre el 75 % de las neuronas

Los resultados obtenidos por las métricas de rendimiento estimulando al 75 % de las neuronas han sido:

Anomaly points: 240
Precision: 0.833
Recall: 1.0
F1-score: 0.909

Esta vez obtenemos las mismas soluciones que con la sobreestimulación al 50 % de las neuronas, es decir, de los 240 puntos detectados como anomalías, 200 son verdaderos positivos, 40 son falsos positivos y el resto son verdaderos negativos. Podríamos pensar que, además de obtener las mismas métricas, los puntos detectados como anomalías también son los mismos. Sin embargo, esto no ocurre así, tal y como vemos en los gráficos de la Figura 22 y la Figura 23.

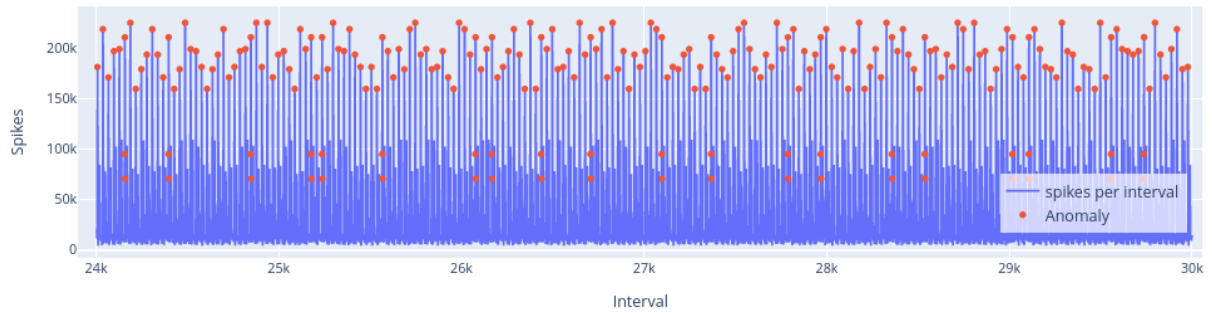


Figura 22: Detección del ataque FLO al 75 % de las neuronas sobre el conjunto de prueba.

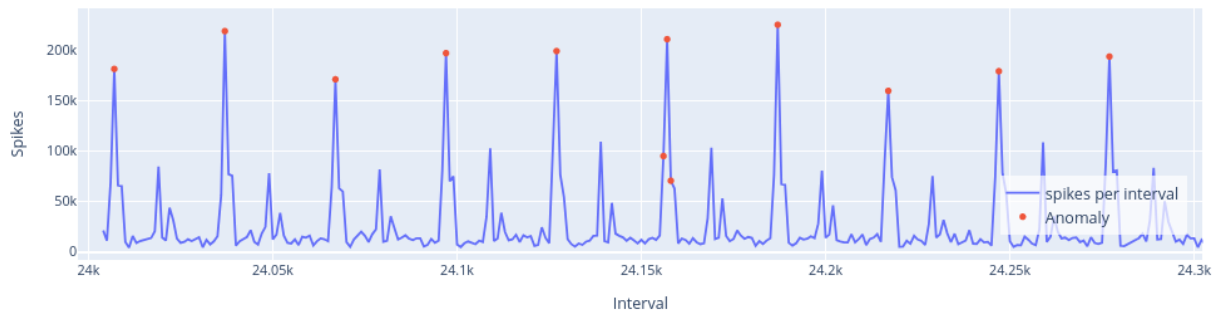


Figura 23: Detección del ataque FLO al 75 % de las neuronas sobre una permutación.

De nuevo, en la primera gráfica observamos que el número de spikes por intervalo en el momento del ataque es mucho mayor que estimulando al 50 % de las neuronas. En cuanto a la segunda gráfica vemos se detectan 12 anomalías, de las cuales los 10 puntos se corresponden con el ataque FLO y las otras 2 (situadas antes y después del sexto ataque) son falsos positivos.

6.4. Resultados del ataque FLO sobre el 100 % de las neuronas

Por último, los resultados obtenidos por las métricas de rendimiento estimulando al 100 % de las neuronas han sido:

Anomaly points: 220

Precision: 0.909

Recall: 1.0

F1-score: 0.952

Como vemos, se detectan 220 puntos como anomalías. Precision toma el valor de 0.909 y recall de 1.0. Por tanto, de los 220 puntos detectados como anomalías, 200 son verdaderos positivos y 20 son falsos positivos. Además, no hay falsos negativos y el resto de puntos son verdaderos negativos. Como vemos, estos resultados son los mejores de las cuatro pruebas realizadas pues, además de detectar todos los ataques, es el que menos falsos positivos predice y el que mayor valor de f1_score presenta.

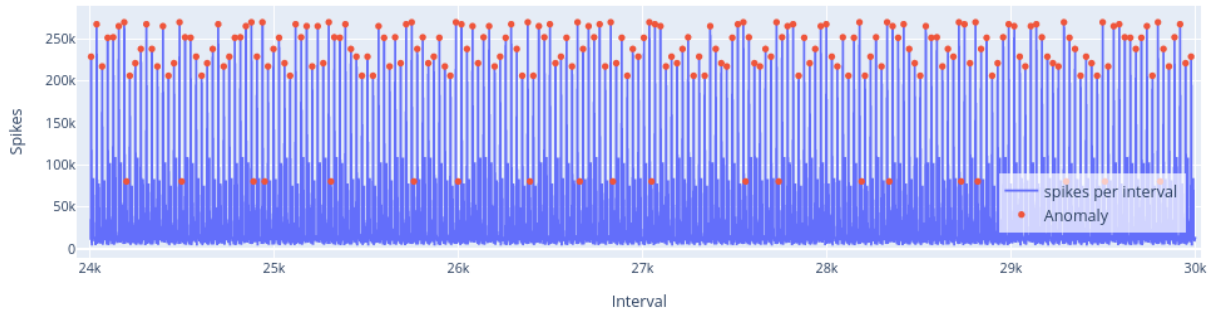


Figura 24: Detección del ataque FLO al 100 % de las neuronas sobre el conjunto de prueba.

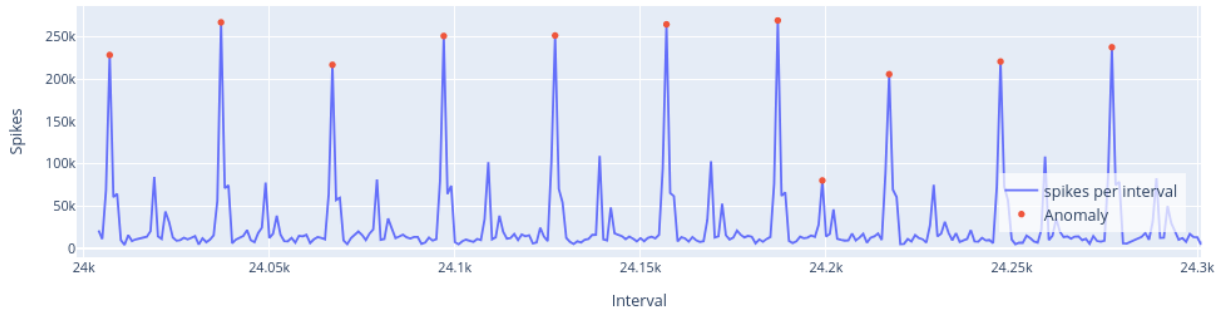


Figura 25: Detección del ataque FLO al 100 % de las neuronas sobre una permutación.

Mostramos de nuevo el número de spikes por intervalo sobre el conjunto de prueba (Figura 24) estimulando, en este caso, a todas las neuronas para que se observe que el número de spikes en el momento del ataque es el mayor de las cuatro pruebas realizadas. Si observamos solo una permutación (Figura 25) vemos que se detectan 11 anomalías, 10 de ellos verdaderos positivos y 1 (el mínimo de los 11) falso positivo.

6.5. Análisis comparativo de los resultados

Una vez analizados los resultados podemos concluir que el modelo LSTM Autoencoder para la detección del ciberataque neuronal FLO presenta unos resultados excelentes (ver Tabla 1), manteniendo un `f1_score` mayor o igual al 90 % en todas las configuraciones de ataque. Además, a medida que aumenta el número de neuronas que se estimulan, los resultados van siendo ligeramente mejores o igual de buenos, tal y como preveíamos al elegir la primera configuración de ataque para decidir la arquitectura de la red.

A partir de la estimulación con el 50 % de las neuronas estimuladas, el modelo consigue predecir el 100 % de los ataques como anomalías, eliminando los falsos negativos. En el caso de la configuración de ataque al 25 %, los ataques son detectados el 90 % de las ocasiones, siendo una cifra bastante alta.

Aunque en todas las configuraciones de ataque se detectan falsos positivos, el hecho de que la precisión se mantenga alta (superior al 80 %) sugiere que este número es manejable. Además, siempre es preferible tratar puntos normales como si fueran anomalías (falsos positivos) a dejar pasar ataques sin detectar (falsos negativos). En ciberseguridad, y más concretamente en el contexto de las BCIs, no detectar un ataque podría tener consecuencias graves en las que la sobreestimulación pudiese causar una alteración de capacidades cognitivas, como puede ser en este caso la visión del ratón. Incluso, podría llegar a causar daños graves de forma irreversible.

Puesto que los resultados son favorables desde la primera configuración y van mejorando ligeramente a medida que se ataca con mayor intensidad, esto sugiere que la capacidad del modelo para detectar ataques es eficiente y robusta, lo cual implica que el modelo es adecuado para detectar ataques al estimular diferentes cantidades de neuronas, no solo las configuraciones actuales. En otras palabras, este modelo podría ser fiable para detectar el ataque FLO cuando se produce sobre un porcentaje considerable de neuronas, aunque de forma bastante variable. Sin embargo, se necesitan mayores esfuerzos para estudiar cómo podrían aplicarse estos modelos a ataques más sutiles en los que solo un pequeño subconjunto de neuronas se ve afectado.

Prueba	Anomalías	TP	FP	FN	Precision	Recall	F1_score
FLO25	200	180	20	20	0.9	0.9	0.9
FLO50	240	200	40	0	0.833	1.0	0.909
FLO75	240	200	40	0	0.833	1.0	0.909
FLO100	220	200	20	0	0.909	1.0	0.952

Tabla 1: Tabla comparativa de los resultados con las distintas configuraciones de ataque.

7. Conclusiones y Vías Futuras

La capacidad de las BCI para mejorar la calidad de vida de las personas es innegable. Estas interfaces permiten una comunicación directa entre el cerebro y dispositivos externos, posibilitando el control de prótesis, el tratamiento de enfermedades neurodegenerativas como el Parkinson o el Alzheimer y la rehabilitación, entre otras. Además, introducen innovaciones en el ámbito de la autenticación, los videojuegos o el entretenimiento, ampliando así su impacto más allá del campo médico.

En este sentido, la importancia de la ciberseguridad en el contexto de las BCIs es crucial. Dada la íntima conexión entre el cerebro y los dispositivos BCI, estos sistemas ofrecen a los atacantes una potencial oportunidad para manipular la actividad cerebral de los usuarios, lo que podría derivar en consecuencias graves, como el robo de la información privada o la manipulación de procesos cognitivos. Sin embargo, además de ser escasos, la mayoría de los trabajos en la literatura científica abordan la ciberseguridad de los sistemas BCI desde una perspectiva teórica. Por ello, la capacidad de predecir y detectar anomalías en la actividad cerebral inducida por ciberataques neuronales representaría un avance significativo en la seguridad de los sistemas BCI.

Debido a esto, este trabajo propone una solución para la identificación de ciberataques neuronales mediante técnicas de deep learning en entornos neuronales simulados. En concreto, se ha desarrollado un modelo basado en una red LSTM Autoencoder para detectar anomalías en la actividad neuronal que indique la presencia del ciberataque neuronal FLO, centrado en sobreestimar un conjunto de neuronas. Este enfoque se ha probado con cuatro configuraciones de ataque según la cantidad de neuronas atacadas correspondientes con el 25 %, 50 %, 75 % y 100 %.

Los resultados obtenidos confirman la capacidad del modelo para identificar de manera precisa el ataque FLO, logrando una detección del 100 % desde la segunda configuración y del 90 % en la primera. A medida que aumenta el grado del ataque, los resultados van mejorando ligeramente, aunque se generan algunos falsos positivos, los cuales son manejables y preferibles a falsos negativos. Esto sugiere que el modelo es adecuado para detectar el ciberataque neuronal FLO en otras configuraciones donde se ataque a un porcentaje considerable de neuronas.

En cuanto al trabajo futuro, se plantea realizar más pruebas para evaluar la eficacia del modelo en la detección del ataque FLO cuando se aplica a un porcentaje pequeño de neuronas, así como explorar la evolución del voltaje de las neuronas en lugar del número de spikes para su detección. Además, dado que solo se ha estudiado uno de los ocho ciberataques neuronales propuestos en la literatura [5], se propone ampliar el alcance del trabajo para la detección de los siete restantes. Por último, se sugiere explorar el uso de otras técnicas o algoritmos de machine learning a parte del modelo LSTM Autoencoder con el fin de mejorar aún más la precisión y robustez en la detección de ataques.

Referencias

- [1] Allen Institute. Biophysical modeling tools (bmtk). <https://alleninstitute.github.io/bmtk/>.
- [2] Ethem Alpaydin. *Introduction to Machine Learning*. MIT Press, 3th edition, 2014.
- [3] BBC. Neuralink: Musk’s firm says first brain-chip patient plays online chess. <https://www.bbc.com/news/business-68622781>, 2022.
- [4] Sergio López Bernal, Alberto Huertas Celdrán, Lorenzo Fernández Maimó, Michael Taynnan Barros, Sasitharan Balasubramaniam, and Gregorio Martínez Pérez. Cyberattacks on miniature brain implants to disrupt spontaneous neural signaling. *IEEE Access*, 8:152204–152222, 2020.
- [5] Sergio López Bernal, Alberto Huertas Celdrán, and Gregorio Martínez Pérez. Eight reasons to prioritize brain-computer interface cybersecurity. *ACM*, page 9, 2022.
- [6] Sergio López Bernal, Alberto Huertas Celdrán, Gregorio Martínez Pérez, Michael Taynnan Barros, and Sasitharan Balasubramaniam. Security in brain-computer interfaces: State-of-the-art, opportunities, and future challenges. *ACM Computing Surveys*, 53(6):1–35, 2020.
- [7] Jason Brownlee. A gentle introduction to lstm autoencoders. <https://machinelearningmastery.com/lstm-autoencoders/>, 2020.
- [8] Koustav Dutta, Rasmita Lenka, Soumya Ranjan Nayak, Asimananda Khandual, and Akash Kumar Bhoi. Med-net: a novel approach to eeg anomaly detection using lstm auto-encoders. *International Journal of Computer Applications in Technology*, pages 343–357, 2021.
- [9] Emotiv. Emotiv | brain data & ai solutions for research and innovation. <https://www.emotiv.com/>.
- [10] Mario Frank, Tiffany Hwu, Sakshi Jain, Robert T. Knight, Ivan Martinovic, Prateek Mittal, Daniele Perito, Ivo Sluganovic, and Dawn Song. Using eeg-based bci devices to subliminally probe for private information. In *Proceedings of the Workshop on Privacy in the Electronic Society (WPES’17)*, pages 133–136, New York, New York, 2017. ACM Press.
- [11] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O’Reilly Media, 2nd edition, 2019.
- [12] Jason J. Han. Synchron receives fda approval to begin early feasibility study of their endovascular, brain-computer interface device. *Artificial Organs*, 2021.

- [13] CJ Hartmann, S Fliegen, SJ Groiss, L Wojtecki, and A Schnitzler. An update on best practice of deep brain stimulation in parkinson's disease. *Therapeutic Advances in Neurological Disorders*, 12, 2019.
- [14] D.M. Hawkins. *Identification of Outliers*. Springer Netherlands, 1980.
- [15] Jennifer Jackson. Everything you need to know about deep brain stimulation (dbs). <https://medium.com/@jenniferjackson1811/everything-you-need-to-know-about-deep-brain-stimulation-dbs-6f520a25dea9>, 2023.
- [16] QianQian Li, Ding Ding, and Mauro Conti. Brain-computer interface applications: Security and privacy challenges. In *Proceedings of the 2015 IEEE Conference on Communications and Network Security (CNS)*, pages 663–666, 2015.
- [17] Susan Li. Time series of price anomaly detection with lstm. <https://towardsdatascience.com/time-series-of-price-anomaly-detection-with-lstm-11a12ba4f6d9>, 2020.
- [18] Pengfei Liu, Xiaoming Sun, Yang Han, Zhishuai He, Weifeng Zhang, and Chenxu Wu. Arrhythmia classification of lstm autoencoder based on time series anomaly detection. *Biomedical Signal Processing and Control*, 71:103228, 2022.
- [19] Stefano Longari, Daniel Humberto Nova Valcarcel, Mattia Zago, Michele Carminati, and Stefano Zanero. Cannolo: An anomaly detection system based on lstm autoencoders for controller area network. *IEEE Transactions on Network and Service Management*, 18(2):1913–1924, 2021.
- [20] Lumen Learning. The Action Potential. <https://courses.lumenlearning.com/suny-ap1/chapter/the-action-potential/>.
- [21] Y. Luo, Y. Sun, X. Tian, X. Zheng, X. Wang, W. Li, X. Wu, B. Shu, and W. Hou. Deep brain stimulation for alzheimer's disease: Stimulation parameters and potential mechanisms of action. *Frontiers in Aging Neuroscience*, March 2021.
- [22] Victoria Magdalena López Madejska, Sergio López Bernal, Gregorio Martínez Pérez, and Alberto Huertas Celdrán. Impact of neural cyberattacks on a realistic neuronal topology from the primary visual cortex of mice. *Wireless Networks*, 2024.
- [23] Ivan Martinovic, Doug Davies, Mario Frank, Daniele Perito, Tomas Ros, and Dawn Song. On the feasibility of Side-Channel attacks with Brain-Computer interfaces. In *21st USENIX Security Symposium (USENIX Security 12)*, pages 143–158, Bellevue, WA, August 2012. USENIX Association.

- [24] Ali H. Mirza and Selin Cosan. Computer network intrusion detection using sequential lstm neural networks autoencoders. In *2018 26th Signal Processing and Communications Applications Conference (SIU)*, pages 1–4, 2018.
- [25] Muse. Muse: the brain sensing headband. <https://choosemuse.com/>.
- [26] Elon Musk and Neuralink. An integrated brain-machine interface platform with thousands of channels. *bioRxiv*, 2019.
- [27] Ali Nawaz, Shehroz S. Khan, and Amir Ahmad. Ensemble of autoencoders for anomaly detection in biomedical data: A narrative review. *IEEE Access*, 12:17273–17289, 2024.
- [28] NEST Simulator. Nest simulator. <https://www.nest-simulator.org/>.
- [29] Christopher Olah. Understanding lstm networks. <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>, 2015.
- [30] Osmosis. Neuron Action Potential. https://www.osmosis.org/learn/es/Neuron_action_potential.
- [31] Pandas Development Team. 10 minutes to pandas. https://pandas.pydata.org/pandas-docs/stable/user_guide/10min.html, 2023.
- [32] Queensland Brain Institute. Action Potentials and Synapses. <https://qbi.uq.edu.au/brain-basics/brain/brain-physiology/action-potentials-and-synapses>.
- [33] Rabie Ramadan and Athanasios Vasilakos. Brain computer interface: Control signals review. *Neurocomputing*, 223:26–44, 2016.
- [34] Neethu Robinson, Ravikiran Mane, Tushar Chouhan, and Cuntai Guan. Emerging trends in bci-robotics for motor control and rehabilitation. *Current Opinion in Biomedical Engineering*, 20:100354, 2021.
- [35] Moumita Roy, Sukanta Majumder, Anindya Halder, and Utpal Biswas. Ecg-net: A deep lstm autoencoder for detecting anomalous ecg. *Engineering Applications of Artificial Intelligence*, 124:106484, 2023.
- [36] Tomas Sabata and Martin Holena. Active learning for lstm-autoencoder-based anomaly detection in electrocardiogram readings, 2020. En línea: https://ceur-ws.org/Vol-2660/ialatecml_shortpaper1.pdf.
- [37] Scribbr. Standard deviation: Interpretations and calculations. <https://www.scribbr.com/statistics/standard-deviation/>.

- [38] Seaborn Development Team. Introduction to seaborn. <https://seaborn.pydata.org/tutorial/introduction.html>, 2023.
- [39] Ruhin Abdulgani Shaikh and S V Shashikala. An autoencoder and lstm based intrusion detection approach against denial of service attacks. In *2019 1st International Conference on Advances in Information Technology (ICAIT)*, pages 406–410, 2019.
- [40] Xiaokang Shu, Shugeng Chen, Guohong Chai, Xinjun Sheng, Jie Jia, and Xiangyang Zhu. Neural modulation by repetitive transcranial magnetic stimulation (rtms) for bci enhancement in stroke patients. *IEEE Engineering in Medicine and Biology Society (EMBC)*, 2018.
- [41] Nina Shvetsova, Bart Bakker, Irina Fedulova, Heinrich Schulz, and Dmitry V. Dylov. Anomaly detection in medical imaging with deep perceptual autoencoders. *IEEE Access*, 9:118571–118583, 2021.
- [42] S. Siuly, Y. Li, and Y. Zhang. *Electroencephalogram (EEG) and Its Background*. Springer, Cham, 2016.
- [43] Synchron. Synchron announces first human u.s. brain-computer interface implant, 2022. En línea.
- [44] Wikipedia. Interfaz cerebro-computadora. https://es.wikipedia.org/wiki/Interfaz_cerebro-computadora.
- [45] Wikipedia. Neurona. <https://es.wikipedia.org/wiki/Neurona>.
- [46] Wikipedia. Neural network (machine learning). [https://en.wikipedia.org/wiki/Neural_network_\(machine_learning\)](https://en.wikipedia.org/wiki/Neural_network_(machine_learning)), 2023.
- [47] Biao Zhang, Jianjun Wang, and Thomas Fuhlbrigge. A review of the commercial brain-computer interface technology from perspective of industrial robotics. pages 379–384, 2010.
- [48] Zhi-Ping Zhao, Chuang Nie, Cheng-Teng Jiang, Sheng-Hao Cao, Kai-Xi Tian, Shan Yu, and Jian-Wen Gu. Modulating brain activity with invasive brain-computer interface: A narrative review. *Brain Sciences*, 13(1):134, 2023.